

Henrique Paes de Souza
Izaque Sena dos Santos
Luiz Guilherme Ramos Flamínio

ULA: Plataforma digital para gerenciamento de pequenos negócios

São Paulo, SP

2025

Henrique Paes de Souza
Izaque Sena dos Santos
Luiz Guilherme Ramos Flamínio

ULA: Plataforma digital para gerenciamento de pequenos negócios

Trabalho de conclusão de curso apresentado ao Departamento de Engenharia de Computação e Sistemas Digitais da Escola Politécnica da Universidade de São Paulo para obtenção do Título de Engenheiro.

Universidade de São Paulo – USP

Escola Politécnica

Departamento de Engenharia de Computação e Sistemas Digitais (PCS)

Orientador: Prof. Dr. Reginaldo Arakaki

São Paulo, SP

2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio convencional ou eletrônico, para fins de estudo e pesquisa, desde que citada a fonte.

Catálogo-na-publicação

Souza, Henrique

ULA: Plataforma digital para gerenciamento de pequenos negócios / H. Souza, I. Santos, L. Flamínio -- São Paulo, 2025.
43 p.

Trabalho de Formatura - Escola Politécnica da Universidade de São Paulo. Departamento de Engenharia de Computação e Sistemas Digitais.

1.Plataformas Digitais 2.Microserviços 3.RM-ODP I.Universidade de São Paulo. Escola Politécnica. Departamento de Engenharia de Computação e Sistemas Digitais II.t. III.Santos, Izaque IV.Flamínio, Luiz

Este trabalho é dedicado à todas as pessoas inquietas, que por algum motivo acreditam veementemente que podem tornar o mundo um lugar melhor através de suas habilidades

Agradecimentos

Agradecemos às nossas famílias pelo apoio incondicional ao longo da graduação. Aos nossos amigos, pela paciência, companheirismo e pelos momentos de descontração que tornaram essa trajetória mais leve e significativa. À Escola Politécnica da Universidade de São Paulo, instituição que nos proporcionou um ambiente de excelência acadêmica e oportunidades ímpares de desenvolvimento profissional. A todos os professores que cruzaram nosso caminho durante a graduação, agradecemos pelos ensinamentos e pela dedicação. Ao nosso orientador, Professor Doutor Reginaldo Arakaki, agradecemos especialmente pela orientação precisa, disponibilidade e pelos valiosos conselhos ao longo do desenvolvimento deste trabalho.

Resumo

O presente trabalho apresenta o desenvolvimento de uma plataforma digital voltada para o gerenciamento de pequenos negócios no Brasil. A ULA oferece funcionalidades visando suprir lacunas na administração e facilitar a tomada de decisão baseada em dados. Os resultados demonstram viabilidade técnica e potencial de impacto social significativo para o ecossistema de pequenos empreendedores brasileiros.

Palavras-chave: plataforma digital. microserviços. RM-ODP.

Abstract

This work presents the development of a digital platform for managing small businesses in Brazil. ULA offers functionalities aiming to fill gaps in administration and facilitate data-driven decision making. Results demonstrate technical feasibility and significant social impact potential for the Brazilian small business ecosystem.

Keywords: digital platform. microservices. RM-ODP.

Lista de ilustrações

Figura 1 – Organização das visões arquiteturais adaptadas do RM-ODP	18
Figura 2 – Diagrama de Osterwalder (Canvas de proposta de valor)	21
Figura 3 – Visão de negócios	27
Figura 4 – Requisitos de engenharia	28
Figura 5 – Correlação das visões de negócios e requisitos de engenharia	29
Figura 6 – Schemas do banco de dados	30
Figura 7 – Tabelas do schema de customers	31
Figura 8 – Tabelas do schema de appointments	32
Figura 9 – Tabelas do schema de financial management	33
Figura 10 – Arquitetura completa da aplicação	37
Figura 11 – Organização de pastas geral	38
Figura 12 – Organização de pastas backend	39
Figura 13 – Microsserviços da plataforma implementados em código	39
Figura 14 – Implementação em código do microsserviço customer	40
Figura 15 – Implementação em código do microsserviço de agendamentos	40
Figura 16 – Implementação em código do microsserviço de gestão financeira	40
Figura 17 – Implementação em código do API Gateway	41
Figura 18 – Implementação em código do frontend	41
Figura 19 – Arquitetura da visão de engenharia	43
Figura 20 – Tela de gerenciamento de funcionários	44
Figura 21 – Modal para cadastrar um novo funcionário	45
Figura 22 – Modal para atualizar dados de um funcionário	46
Figura 23 – Modal para deletar um funcionário	46
Figura 24 – Modal para associar serviços a funcionários	47
Figura 25 – Tela de gerenciamento de serviços	47
Figura 26 – Modal para cadastrar um novo serviço	48
Figura 27 – Modal para atualizar dados de um serviço	49
Figura 28 – Modal para deletar um serviço	49
Figura 29 – Tela de gerenciamento de clientes	50
Figura 30 – Modal para atualizar o nome de um cliente	50
Figura 31 – Exemplo de conversa no WhatsApp	51
Figura 32 – Fluxo da aplicação com uso do WhatsApp	51
Figura 33 – Visualização completa da agenda	52
Figura 34 – Modal para cadastrar um bloqueio na agenda	52
Figura 35 – Modal de agendamento de serviço	53

Figura 36 – Componentes principais de contas a pagar	54
Figura 37 – Modal de registro de nova conta a pagar	54
Figura 38 – Componentes principais de contas a receber	54
Figura 39 – Principais componentes da página de despesas	55
Figura 40 – Principais componentes da página de entradas	55
Figura 41 – Dashboard de saúde financeira - controle de granularidade e insights . .	56
Figura 42 – Dashboard de saúde financeira - previsão de variação de caixa	56
Figura 43 – Dashboard de saúde financeira - visão geral dos elementos que compõem a saúde financeira	56
Figura 44 – Seleção de serviços para agendamento	58
Figura 45 – Escolha da data para agendar os serviços	59
Figura 46 – Escolha do profissional que irá realizar o serviço	60
Figura 47 – Escolha de um horário disponível	61
Figura 48 – Resumo do agendamento atual	62

Lista de abreviaturas e siglas

AES	Advanced Encryption Standard
API	Application Programming Interface
AWS	Amazon Web Services
B2B2C	Business to Business to Consumer
BMC	Business Model Canvas
CNPJ	Cadastro Nacional da Pessoa Jurídica
CSS	Cascading Style Sheets
ECB	Electronic Codebook
gRPC	Google Remote Procedure Call
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
MEI	Microempreendedor Individual
MPE	Micro e Pequenas Empresas
RM-ODP	Reference Model for Open Distributed Processing
REST	Representational State Transfer
Sebrae	Serviço Brasileiro de Apoio às Micro e Pequenas Empresas
URL	Uniform Resource Locator
VPC	Virtual Private Cloud
GEM	Global Entrepreneurship Monitor

Sumário

1	INTRODUÇÃO	12
1.1	Motivação	12
1.2	Justificativa	12
1.3	Organização do Trabalho	13
2	FUNDAMENTOS CONCEITUAIS	15
2.1	Pequenas empresas no Brasil	15
2.2	Plataformas digitais para pequenas e médias empresas	16
2.3	ISO 10746 - Visões Arquiteturais	16
3	ESPECIFICAÇÃO DO PROJETO	19
3.1	Escopo	19
3.1.1	Tarefas dos clientes	19
3.1.2	Dores dos clientes	19
3.1.3	Ganhos dos clientes	20
3.1.4	Produtos e serviços	20
3.1.5	Aliviadores de dores	20
3.1.6	Criadores de ganhos	20
3.1.7	Diagrama de Osterwalder da plataforma	20
3.2	Especificação de requisitos	21
3.2.1	Cadastro	21
3.2.1.1	Casos de uso	22
3.2.1.2	Requisitos funcionais	22
3.2.1.3	Requisitos não funcionais	22
3.2.2	Agendamento	22
3.2.2.1	Casos de uso	23
3.2.2.2	Requisitos funcionais	23
3.2.2.3	Requisitos não funcionais	23
3.2.3	Gestão financeira	24
3.2.3.1	Casos de uso	24
3.2.3.2	Requisitos funcionais	24
3.2.3.3	Requisitos não funcionais	25
3.2.4	Dashboard de saúde financeira	25
3.2.4.1	Casos de uso	25
3.2.4.2	Requisitos funcionais	25
3.2.4.3	Requisitos não funcionais	26

3.3	Visão de negócios	26
4	RESULTADOS OBTIDOS	30
4.1	Visão de informação	30
4.2	Visão computacional	35
4.3	Visão tecnológica	37
4.4	Visão de Engenharia	42
4.5	Telas e experiência do usuário	43
5	CONSIDERAÇÕES FINAIS	63
5.1	Conclusões do Projeto de Formatura	63
5.2	Perspectivas de continuidade	64
5.3	Do curso de engenharia	65
	REFERÊNCIAS	67

1 Introdução

1.1 Motivação

O presente documento tem por objetivo apresentar o projeto de formatura do curso de Engenharia de Computação da Escola Politécnica da Universidade de São Paulo, que será realizado pelo grupo composto por Henrique Paes de Souza, Izaque Sena dos Santos e Luiz Guilherme Ramos Flamínio orientados pelo Professor Doutor Reginaldo Arakaki. Nesse documento, mostra-se uma solução de engenharia baseada em Plataforma Digital aplicada em um contexto real visando resolver um problema do cotidiano e com valor social agregado utilizando técnicas aprendidas durante o curso, alinhadas com experiências adquiridas em atividades extra-curriculares. Devido à amplitude do tema, diversas ideias surgiram no grupo e foram discutidas a fundo para verificar a viabilidade. Para a verificação, definiram-se alguns critérios para a seleção da Plataforma Digital a ser construída, sendo eles:

- Alinhamento com as necessidades de uma parcela da população.
- Potencial de ser continuada como uma empresa após a entrega do trabalho de formatura.
- Impacto social esperado com o produto final.

Diante dos critérios estabelecidos, optou-se pela elaboração de uma plataforma digital para gerenciamento de pequenos negócios, visando simplificar problemas do domínio relacionados à gestão de finanças e agenda de um pequeno negócio, com potencial de impactar simultaneamente empreendedores e clientes.

A formação do grupo se deu inicialmente pelo mútuo interesse na construção de uma aplicação no estilo Plataforma Digital focada na resolução de um problema real da sociedade.

1.2 Justificativa

O termo Plataforma Digital tem sido amplamente explorado na academia e muito discute-se sua definição, sendo que esta depende do ponto de vista adotado ([ASADULLAH; FAIK; KANKANHALLI, 2023](#)). Do ponto de vista de negócios, uma plataforma digital é um ecossistema suportado por estrutura computacional cujo valor oferecido se encontra na facilitação da integração entre grupos de usuários, por exemplo, pequenos empreendedores e clientes ou fornecedores e consumidores ([GIBSON, 2024](#)).

Os avanços recentes na área de tecnologia, como o advento da computação em nuvem e a criação de ferramentas para análise de Big Data fomentam o surgimento de diversas novas plataformas digitais (HEIN et al., 2020), que por sua vez mudam consideravelmente a dinâmica do mercado e a forma de consumo de bens e serviços por parte dos usuários. Tal mudança de dinâmica faz com que as empresas despriorizem o domínio da cadeia de valor, em favor da capacidade de atrair os fornecedores de serviço para suas plataformas (REUVER; SØRENSEN; BASOLE, 2018).

Por exemplo, um estudo da Business Insider de 2017 mostrou que a plataforma digital do Airbnb contava com 4 milhões de imóveis listados para locação, superando as acomodações oferecidas pelas cinco maiores redes de hotéis mundiais somadas (HARTMANS, 2017). A partir do entendimento da mudança de paradigma no cenário corporativo atual, pode-se afirmar que as plataformas digitais são responsáveis pela integração de diversos ecossistemas de negócios, conectando não apenas os usuários aos resolvedores de suas dores, mas também conectando diferentes empresas e oferecendo soluções amplamente integradas no modelo B2B2C.

Pesquisas estimam que, até 2027, os investimentos de empresas com transformação digital atingirão a marca de U\$3,9 trilhões, explicitando assim a importância deste mercado para o cenário mundial (GIBSON, 2024).

Alinhado aos aspectos de mercado apresentados neste capítulo, a possibilidade de exercício das habilidades obtidas durante a graduação em Engenharia da Computação também foi um fator determinante para a escolha do tema. Isso deve-se ao fato de que a construção de uma plataforma digital engloba diversos temas da engenharia que poderiam ser praticados durante a implementação do projeto, dentre eles:

- Integração eficiente entre módulos distribuídos.
- Arquitetura tolerante a falhas.
- Rastreabilidade.
- Integração com sistemas externos

A partir dos fatos apresentados, definiu-se o tema de *Plataforma Digital para Gerenciamento de Pequenos Negócios*, que será explorado na continuação do presente documento.

1.3 Organização do Trabalho

O Trabalho é dividido em 4 partes principais, inicialmente o Capítulo 2 apresenta métricas do mercado em que se situa a solução da plataforma digital proposta e apresenta a ISO 10746 que foi utilizada para definir como analisar o trabalho proposto, partindo

para o Capítulo 3 que destrincha a solução proposta através dos requisitos propostos e as dores que são resolvidas através da plataforma, que por sua vez são consolidados no Capítulo 4 exibindo os resultados das telas implementadas e explicando a plataforma obtida de acordo com as fundamentações teóricas estabelecidas. Por fim, o Capítulo 5 traz as considerações finais quanto ao projeto do ponto de vista da implementação e engenharia mostrando inclusive os próximos passos.

2 Fundamentos Conceituais

2.1 Pequenas empresas no Brasil

Hoje no Brasil existem mais de 3,7 milhões de empresas, sendo que destas 96% (3,5 milhões) são pequenos negócios – microempreendedores individuais (MEI), microempresas e empresas de pequeno porte (MPE) ([ABREU, 2024](#)). Dados do Serviço Brasileiro de Apoio às Micro e Pequenas Empresas (Sebrae) apontam que os pequenos negócios movimentam cerca de R\$ 420 bilhões por ano, valor que corresponde a aproximadamente 30% do PIB brasileiro ([DIAS, 2024](#)).

Além disso, uma edição da pesquisa Monitor Global de Empreendedorismo (Global Entrepreneurship Monitor – GEM) mostrou que, em 2023, o Brasil somou 90 milhões de empreendedores ou candidatos a empreendedores, sendo 42 milhões de adultos que já tinham um negócio ou que fizeram alguma ação em 2023 visando ter um negócio no futuro, e 48 milhões de adultos que não têm um empreendimento, mas que gostariam de ter um em até 3 anos. Esses números colocam o Brasil em 2º lugar no ranking de estimativa absoluta de empreendedores potenciais ([UM. . . , 2024](#)).

Apesar dos números muito promissores a respeito do assunto, a pesquisa do Sebrae sobre a sobrevivência dessas empresas apontou que mais de 40% das empresas não sobrevivem mais do que 5 anos, principalmente devido a: falta de planejamento, despreparo na liderança e descontrole na gestão ([EMPREENDEDORISMO. . . , 2024](#)).

Este número explicita as dores dos donos de pequenos negócios em relação à administração de suas empresas, sendo um domínio com potencial para a elaboração de um mecanismo para a solução desse problema. Ao conversar brevemente com donos de negócios frequentados pelo grupo para entender mais a fundo os problemas de gestão e suas causas, observou-se uma vasta gama de processos manuais, realizados de forma antiquada.

Por exemplo, para agendar uma sessão do serviço ofertado, pode ser necessário ir presencialmente ao local ou então entrar em contato por mensagem diretamente com o dono do serviço para perguntar sobre a disponibilidade. Outro processo que ainda conta com a execução manual dos empreendedores é o controle financeiro, por exemplo, fluxo de caixa ou folha de pagamento, de forma que os donos de negócio não possuem respaldo para tomar decisões vitais para a saúde de suas empresas, levando-as ao fechamento, conforme foi apontado anteriormente.

A existência de um grande número de processos manuais pode explicar os problemas gerenciais, por serem muito suscetíveis a erros humanos e dificultarem a organização,

trazendo consequências graves para o negócio, podendo causar, inclusive, o fechamento dessas empresas. Tais dores evidenciam a plataforma como um mecanismo educacional, que não só auxilia o pequeno empreendedor com tarefas do seu dia a dia, mas também é uma fonte confiável de conhecimento em administração e contabilidade.

2.2 Plataformas digitais para pequenas e médias empresas

A plataforma digital apresentada nesta entrega visa automatizar alguns destes processos, facilitando o agendamento e oferecendo ao pequeno empreendedor suporte para a tomada de decisão financeiras com dados. Um levantamento foi realizado pelo grupo via formulário do Google e entrevistas presenciais para entender mais a fundo os problemas que esse projeto poderia solucionar, buscando assim as principais funcionalidades que deveriam ser implementadas, sendo estas:

- Cadastro do estabelecimento, funcionários e de todos os serviços oferecidos no estabelecimento
- Cadastro de clientes (automático via chatbot e manual)
- Controle de agenda configurável via chatbot conectado ao WhatsApp
- Controle de fluxo de caixa
- Dashboard de saúde financeira

Do ponto de vista do cliente, é possível utilizar um chatbot integrado à plataforma do dono para realizar o agendamento (em detrimento do contato direto com os funcionários). Subsequentemente, visa-se realizar uma análise das funcionalidades definidas em termos de proposta de adição de valor através de uma metodologia bem definida.

2.3 ISO 10746 - Visões Arquiteturais

O Modelo de Referência para Processamento Distribuído Aberto (RM-ODP - Reference Model for Open Distributed Processing), padronizado como ISO 10746 ([ISO/IEC, 2009](#)) é um framework conceitual que emergiu devido aos desafios de heterogeneidade, interoperabilidade e escalabilidade em ambientes computacionais distribuídos ([RAYMOND, 1995](#)), estabelecendo princípios arquiteturais que permanecem relevantes para o desenvolvimento de plataformas digitais modernas.

O framework estabelece quatro objetivos centrais para sistemas de processamento distribuído: portabilidade de aplicações, interoperabilidade entre sistemas ODP garantindo a troca robusta de mensagens, ocultação de complexidades da distribuição, e independência de plataforma e tecnologia ([JEBBAR; SEKKAKI; BENAMAR, 2012](#)). Para explicar

sistemas complexos, o modelo estabelece uma abordagem multi-perspectiva que decompõe o sistema em 5 pontos de vista arquiteturais, fornecendo uma semântica formal para descrição de sistemas distribuídos de processamento. Os cinco pontos de vista do sistema são:

- **Ponto de vista empresarial (Enterprise Viewpoint):** estabelece o propósito e escopo de existência do sistema em um contexto organizacional, estabelecendo os objetos empresariais, abstrações representando entidades de negócio e suas relações (STEEN; DERRICK, 2000).
- **Ponto de vista de informação (Information Viewpoint):** representa as entidades de informação do sistema em termos de sua estrutura e organização, bem como as operações que manipulam e transformam a informação (RAYMOND, 1995).
- **Ponto de vista computacional (Computational Viewpoint):** permite a decomposição funcional do sistema em serviços que interagem através de interfaces bem definidas. Os componentes dessa visão são os objetos computacionais, unidades de distribuição com comportamento encapsulado.
- **Ponto de vista de engenharia (Engineering Viewpoint):** define os mecanismos e funções de infraestrutura necessários para suportar as interações distribuídas entre os elementos computacionais. Os componentes estabelecidos são responsáveis por garantir a comunicação e prover transparência de distribuição (JEBBAR; SEKKAKI; BENAMAR, 2012).
- **Ponto de vista tecnológico (Technology Viewpoint):** aborda as escolhas tecnológicas específicas de hardware e software e decisões de padrão de projeto utilizadas na implementação do sistema.

Com os pontos de vista definidos pelo modelo, pode-se obter uma visão mais ampla do projeto, com ganho de facilidade na etapa de design da arquitetura.

Durante o desenvolvimento deste trabalho, utilizou-se como base o modelo definido na ISO 10746 com algumas adaptações para refletir a engenharia de requisitos e as técnicas de desenvolvimento de produtos. Dessa forma, as visões definidas pelo nosso grupo seguem o seguinte diagrama:

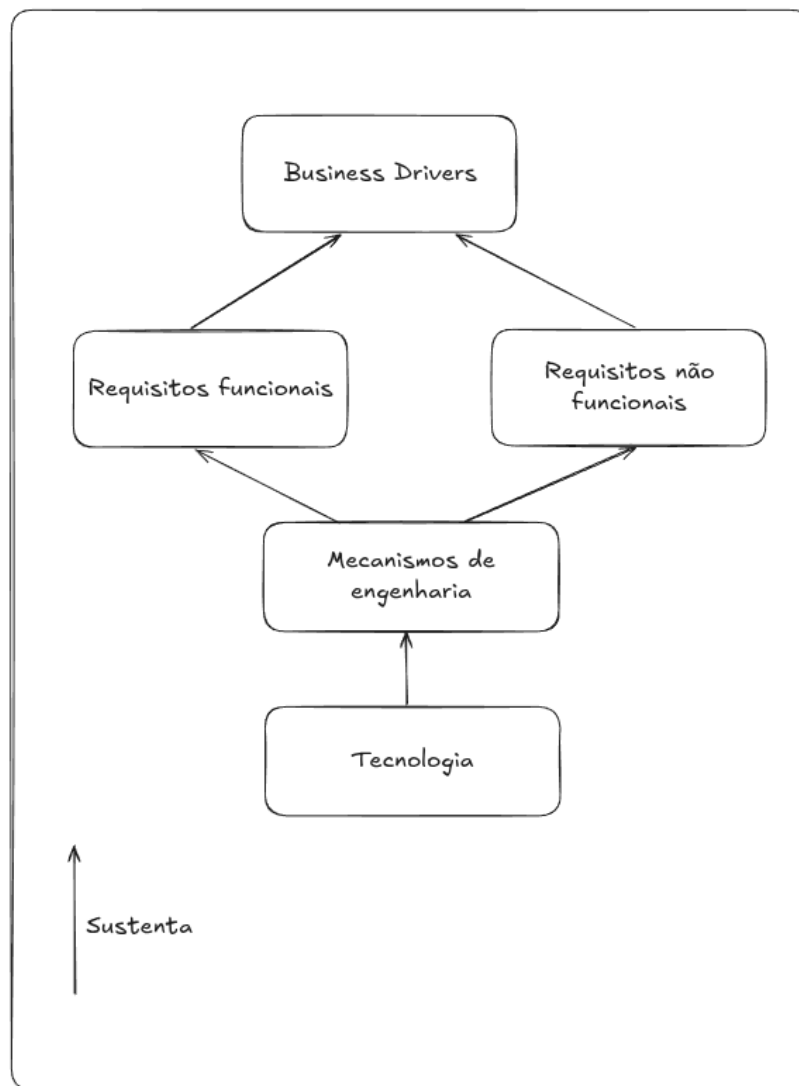


Figura 1 – Organização das visões arquiteturais adaptadas do RM-ODP

Nessa organização, a visão computacional definida originalmente no RM-ODP é dividida entre requisitos funcionais e requisitos não funcionais, de forma a detalhar melhor o resultado esperado da atuação do sistema. Nessa arquitetura, o ponto de vista da informação é tratado em conjunto dos *business drivers* no sentido que as informações operadas pelo sistema são definidas pelas especificidades do modelo de negócio. Destaca-se ainda que as inovações propostas pelo projeto (automatização de fluxos, previsibilidade de caixa, elementos didáticos, dentre outros) são os responsáveis pela concretização dos *business drivers* propostos, e que tais aspectos guiam a definição de requisitos não funcionais, que será explorada adiante no texto.

3 Especificação do Projeto

3.1 Escopo

Alexander Osterwalder, criador do Business Model Canvas (BMC) e seguidor do Lean Startup inventou também outro diagrama com dois influentes nomes do segmento, Yves Pigneur e Alan Smith, visando criar uma ferramenta capaz de acompanhar o processo de elaboração de um sistema, do ponto de vista empresarial, permitindo a equipe projetista mapear e testar a proposta de valor da empresa em relação às necessidades dos clientes, assim através dessa ferramenta a equipe poderia analisar, refletir e adaptar a proposta de valor por conseguir enxergar sinteticamente as dores de seu cliente e as soluções que as empresas oferecem (OSTERWALDER; PIGNEUR, 2003).

Essa ferramenta ficou conhecida como Canvas da Proposta de Valor que atualmente é tida como um complemento do Business Model Canvas pois ela alinha o produto ao mercado, ou seja, ele é usado para refinar o produto descrito focando em dois blocos dos nove existentes no Business Model Canvas, proposta de valor de segmento de clientes, analisando esse encaixe do ponto de vista do mercado. Dentro do Lean Startup ele atua no chamado "encaixe mercado-produto" (NARDES; MIRANDA, 2014).

Para alcançar esse objetivo de encaixe serão analisados do lado do cliente suas tarefas, dores e ganhos e do lado da empresa seus produtos e serviços, aliviadores e criadores de ganhos.

3.1.1 Tarefas dos clientes

Esse primeiro passo consiste em definir o perfil do cliente, descrever o que estão tentando realizar e seus problemas. Para atingir as melhores respostas é necessário conversar de fato com os clientes perguntando as tarefas diárias e necessidades básicas até traçar um perfil inicial.

3.1.2 Dores dos clientes

O segundo passo para completar o diagrama trata das situações indesejadas e experiências negativas atreladas às suas tarefas. Ainda é necessário a conversa com os clientes para descobrir onde ele perde tempo, onde ele não atinge com excelência suas metas de serviços, como as tecnologias atuais não os satisfazem e suas principais barreiras para expansão e evolução.

3.1.3 Ganhos dos clientes

Nessa etapa serão descritos os benefícios que os clientes esperam ou desejam com a solução que a empresa ou sistema poderia oferecer, do ponto de vista funcional, social e emocional, incluindo redução dos custos e dores. E aqui se encerra a conversa com o cliente, onde já deve-se ter traçado o perfil do cliente.

3.1.4 Produtos e serviços

Com o perfil do cliente traçado se volta à proposta de valor questionando a si mesmo, e a equipe, listando todos os produtos e serviços que a empresa ou sistema visa oferecer aos clientes para ajudá-los em suas tarefas. Aqui serão descritos produtos tangíveis, virtuais e até financeiros.

3.1.5 Aliviadores de dores

Tendo os produtos e serviços definidos deve-se pensar como eles criam valor resolvendo as dores dos clientes de forma a reduzir ou até mesmo eliminar as emoções negativas e situações indesejadas além dos riscos que já foram vivenciados ou ainda persistem.

3.1.6 Criadores de ganhos

Por fim, será necessário descrever como os produtos elaborados criam ganhos aos seus clientes, visando o que por eles foi colocado como desejo, mas também é interessante que os surpreendam positivamente incluindo mais funcionalidades ou ganho social, além da redução de custo.

3.1.7 Diagrama de Osterwalder da plataforma

A partir da análise feita considerando os aspectos apresentados nos itens anteriores, é possível elaborar o seguinte Diagrama de Osterwalder da plataforma:

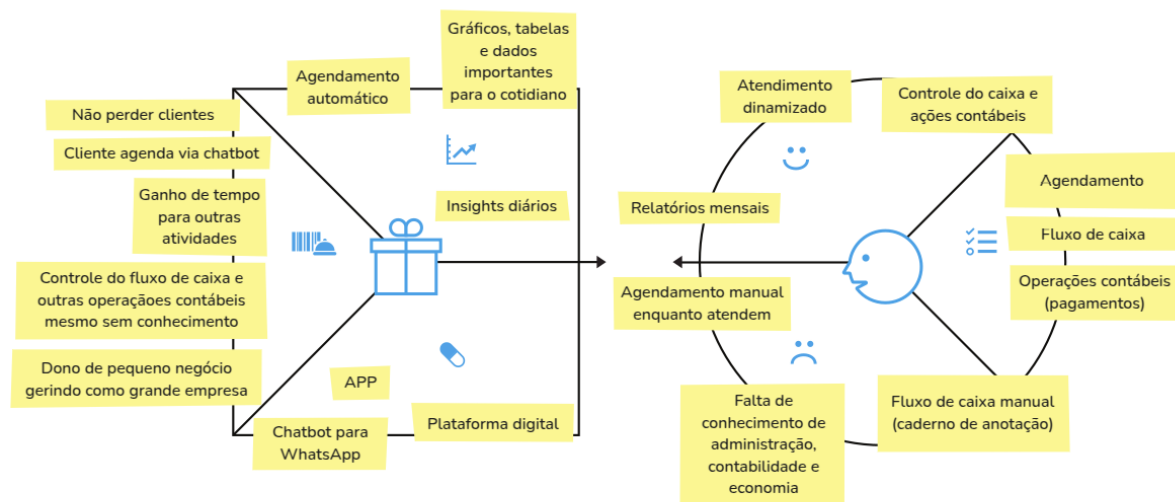


Figura 2 – Diagrama de Osterwalder (Canvas de proposta de valor)

O diagrama explicita que aquelas que foram definidas como dores dos clientes no capítulo atual são efetivamente mitigadas pelas funcionalidades definidas em 2.2, que também aparecem na visualização definida por Osterwalder.

3.2 Especificação de requisitos

A plataforma digital para pequenos empreendedores é especificada no presente documento como o conjunto de requisitos funcionais e não funcionais estabelecidos para as funcionalidades detalhadas anteriormente, seguindo princípios consolidados de engenharia de software (PRESSMAN; MAXIM, 2014), bem como a descrição de seu fluxo de funcionamento. Os requisitos não funcionais definem atributos de qualidade do sistema, seguindo o modelo de qualidade estabelecido pela norma ISO/IEC 25010 (ISO/IEC, 2011), que especifica características como desempenho, segurança, usabilidade e compatibilidade.

3.2.1 Cadastro

O cadastro consiste na funcionalidade de inserção de dados fundamentais dos empreendimentos por um usuário devidamente autenticado na plataforma. Os dados cadastrais podem ser divididos da seguinte forma:

- Dados do empreendimento: nome, ramo de atuação, telefone de contato, localização
- Dados dos serviços: serviços prestados, juntamente de seus respectivos valores e tempos de duração

- Dados dos funcionários: nome, email e telefone
- Dados dos clientes: nome e telefone

O cadastro de clientes será realizado automaticamente quando o mesmo entrar em contato com o número cadastrado através de uma aplicação que conecta a plataforma e a API do aplicativo de mensagens, WhatsApp.

3.2.1.1 Casos de uso

- Cadastrar empreendimento, serviços e funcionários em fluxo manual
- Cadastrar cliente em fluxo automático

3.2.1.2 Requisitos funcionais

- Cadastrar empreendimento
- Cadastrar serviços
- Cadastrar funcionários
- Cadastrar clientes
- Cadastro, edição e exclusão de registros
- Validação de dados
- Login
- Associação de funcionários a serviços
- Agenda de funcionários
- Permissionamento de funcionários (acessos dentro da plataforma)

3.2.1.3 Requisitos não funcionais

- Desempenho: a gestão de informações cadastrais deve ser rápida
- Segurança: apenas usuários devidamente autenticados devem ser capazes de realizar o gerenciamento
- Usabilidade: o gerenciamento deve ser intuitivo e rápido, com poucos cliques

3.2.2 Agendamento

O agendamento consiste na escolha de um horário, no qual o serviço oferecido por um funcionário será prestado para atender um cliente. Logo, o agendamentos devem seguir a opção do cliente conforme a disponibilidade do funcionário. Uma vez que o agendamento

foi efetuado, existe a necessidade da gestão de seu ciclo de vida, podendo ser concluído (gerando caixa), remarcado (gerando um novo agendamento) ou cancelado.

O agendamento, portanto, em nossa plataforma digital consistirá em duas interfaces, uma para o funcionário do empreendimento para gerenciar sua agenda, e outra para o cliente que deseja realizar um agendamento.

A interface de agendamento para os funcionários consistirá em uma agenda que apresentará todos os agendamentos do dia e da semana para cada funcionário, todos os horários indisponíveis dos funcionários, a opção de adicionar agendamentos manuais, cancelar agendamentos prévios, adicionar indisponibilidades, entre outros.

A interface de agendamento para os clientes do empreendimento consistirá em uma página web onde o usuário pode escolher os serviços desejados, data e horário e o funcionário. O link para esta interface é obtido ao interagir com o WhatsApp da plataforma.

3.2.2.1 Casos de uso

- Cliente realiza agendamento pelo link enviado por WhatsApp
- Gerência de agenda feita pelo funcionário

3.2.2.2 Requisitos funcionais

- Envio de mensagens automáticas de confirmação
- Cadastro de agenda
- Funcionário gere sua agenda (marca os dias e horários disponíveis e indisponíveis)
- Funcionário agenda cliente manualmente
- Funcionário deve conseguir visualizar os atendimentos do dia
- Funcionário deve gerenciar ciclo de vida: iniciar, concluir, cancelar ou remarcar um atendimento
- Coleta de informações automáticas para registros

3.2.2.3 Requisitos não funcionais

- Segurança: apenas usuários devidamente autenticados devem poder gerenciar a agenda
- Disponibilidade: o sistema de agendamento deve ser redundante para garantir que o usuário consiga gerenciar a agenda quando necessário

- Usabilidade: as interfaces de agendamento devem ser claras e todas as ações perceptíveis ao acessar a tela
- Atomicidade do ciclo de vida do agendamento: os agendamentos cadastrados não podem ser duplicados conforme mudam de status

3.2.3 Gestão financeira

Gestão financeira é o conjunto de funcionalidades que busca automatizar o controle, gestão e análise financeira do empreendimento. A gestão financeira ocorre principalmente por meio da funcionalidade de fluxo de caixa, o qual permite ao sistema armazenar todas as movimentações financeiras que ocorrem na empresa.

O fluxo de caixa divide-se em duas operações: entrada de caixa e saída de caixa. A entrada de caixa é a operação dentro do fluxo de caixa que engloba todos os ativos financeiros que entraram no empreendimento. Em outras palavras, são todas as operações financeiras que compõem a receita do empreendimento.

As saídas de caixa podem ser divididas em dois tipos: despesa e contas a pagar. Entende-se como despesa, as saídas de caixa avulsas (não recorrentes). As contas a pagar são todas as operações de saída que são recorrentes ao empreendimento. Uma conta a pagar é naturalmente recorrente, e pode ter um estado de não paga ou de paga, sendo que a cada período uma nova conta a pagar é criada com status de não paga.

3.2.3.1 Casos de uso

- Realizar uma entrada de caixa
- Adicionar despesas
- Adicionar contas a pagar
- Adicionar contas a receber

3.2.3.2 Requisitos funcionais

- Adicionar uma entrada de caixa com os serviços realizados por um funcionário
- O valor de uma entrada de caixa deve ser contabilizado positivamente
- Visualizar o histórico de entradas de caixa
- Adicionar uma despesa
- Visualizar históricos de despesas
- Visualizar contas já pagas, e contas a pagar do mês

- O valor de uma despesa ou uma conta de pagar já paga deve ser contabilizada como saída de caixa
- Adicionar uma conta a receber
- Visualizar contas recebidas do mês e contas a receber

3.2.3.3 Requisitos não funcionais

- Usabilidade: Interface intuitiva para facilitar a inserção de dados
- Performance: Processamento rápido de cálculos e registros
- Segurança: Acesso restrito a usuários autorizados
- Confiabilidade: Prevenção de registros duplicados ou inconsistentes
- Auditabilidade: Registro de todas as operações com rastreabilidade
- Disponibilidade: Funcionamento contínuo do sistema

3.2.4 Dashboard de saúde financeira

Os usuários devem ser capazes de acessar relatórios sobre a saúde financeira de seu negócio em forma de dashboards com gráficos e dados agregados sobre o tema, bem como recomendações educacionais sobre como gerenciar a situação financeira exibida. No dashboard, é possível analisar informações acerca dos elementos constituintes da saúde financeira: contas a pagar, contas a receber, entradas e saídas. A exibição desses dados fornece uma visão holística da situação financeira, mostrando como o caixa deve variar no período e quais elementos atualmente influenciam mais a situação financeira da empresa.

3.2.4.1 Casos de uso

- Visualizar relatório financeiro completo
- Controlar a granularidade dos relatórios

3.2.4.2 Requisitos funcionais

- Acesso dos relatórios
- Controle de granularidade dos relatórios
- Exibição de dicas financeiras sobre o cenário atual

3.2.4.3 Requisitos não funcionais

- Acessibilidade cognitiva: o dashboard deve usar termos fáceis e explicar a visão geral do cenário
- Confiabilidade: os elementos exibidos no dashboard devem estar sincronizados com a camada de persistência do sistema
- Usabilidade: o entendimento das informações do dashboard não pode exigir muitos cliques

3.3 Visão de negócios

Define-se a arquitetura do sistema de acordo com cinco visões distintas, de forma que cada uma das visões destaca um aspecto diferente do sistema (BASS; CLEMENTS; KAZMAN, 2021), e cada um dos aspectos por sua vez influencia nas decisões tecnológicas ao longo do desenvolvimento, conforme definido pelo paradigma de táticas arquiteturais (MÁRQUEZ; ASTUDILLO; KAZMAN, 2023).

De acordo com o modelo arquitetural definido no capítulo anterior, duas das visões são apresentadas de forma difusa durante a exibição das features, sendo elas a visão dos requisitos funcionais e as visões dos requisitos não funcionais, detalhadas em capítulos anteriores. As demais visões são exploradas adiante.

A visão de negócios pode ser observada no diagrama abaixo:

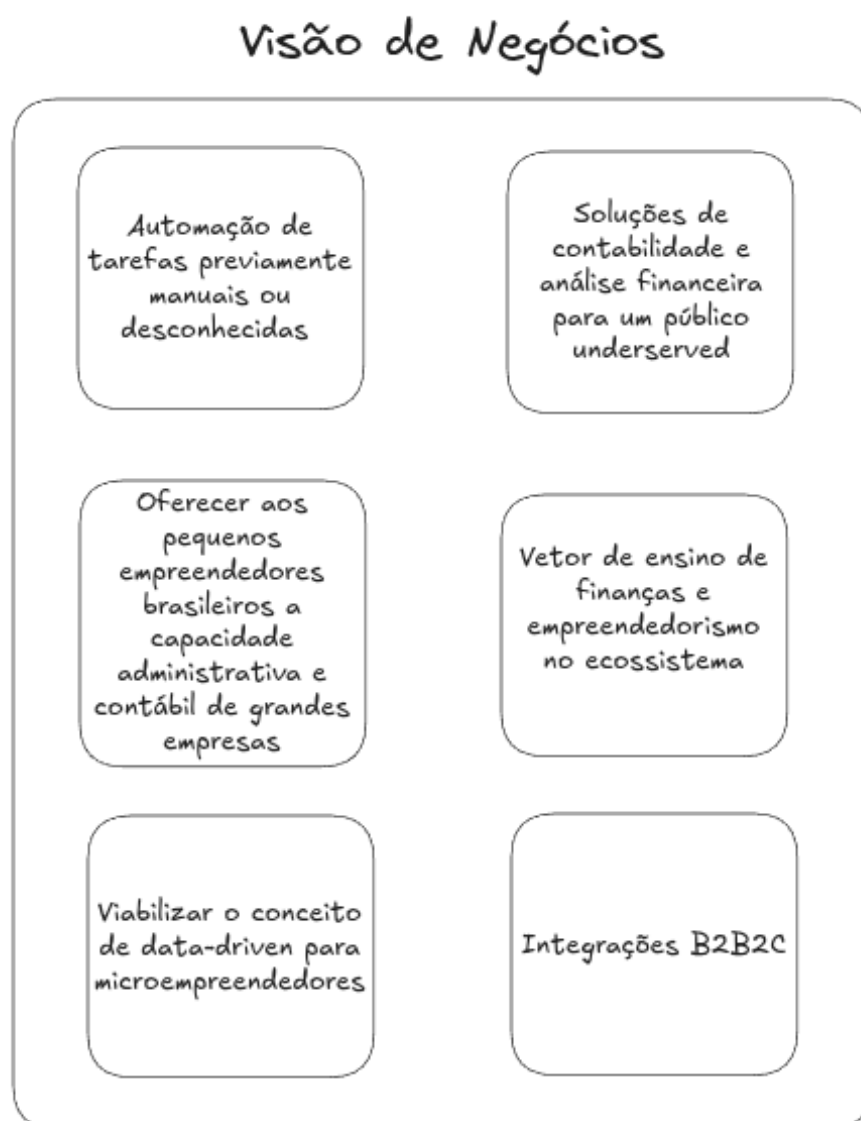


Figura 3 – Visão de negócios

Conforme citado anteriormente, cada uma das visões de negócio implica em decisões tecnológicas a serem feitas adiante no projeto.

Tais decisões representam os requisitos de engenharia para o projeto, que podem ser descritos como:



Figura 4 – Requisitos de engenharia

Destaca-se que, neste ponto é possível correlacionar as visões de negócio e os requisitos de engenharia definidos, de forma a refinar as decisões tecnológicas do projeto, para que atinjam a visão de negócios inicialmente definida:

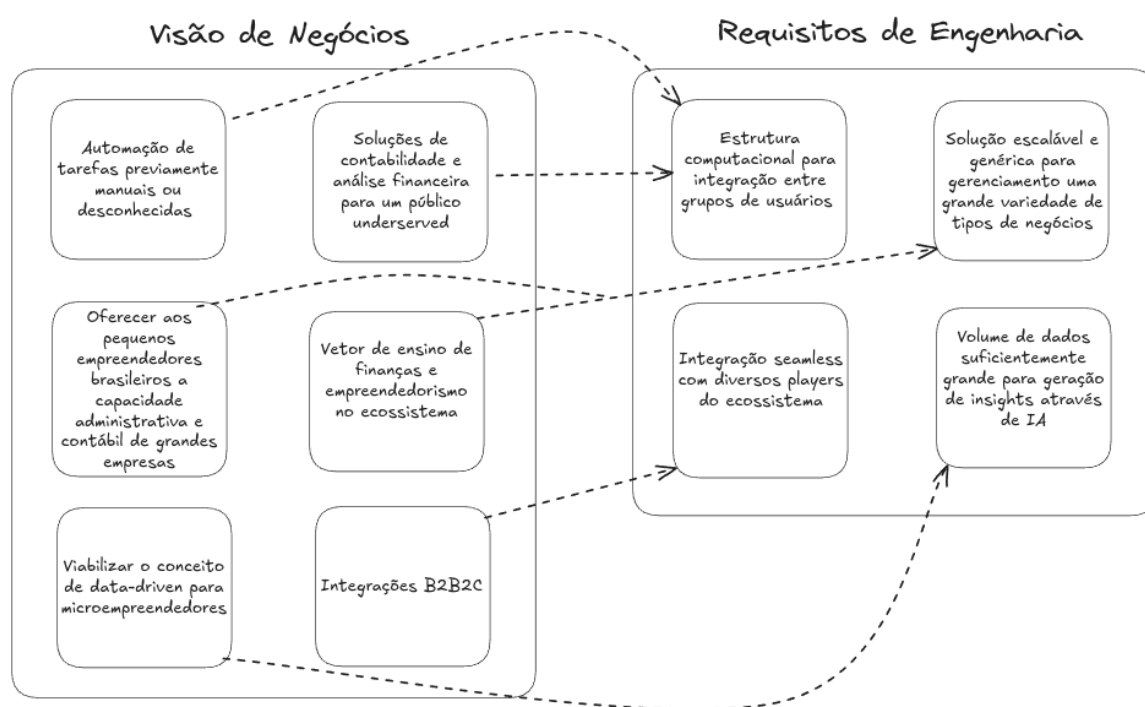


Figura 5 – Correlação das visões de negócios e requisitos de engenharia

4 Resultados Obtidos

4.1 Visão de informação

Após a análise de requisitos funcionais e de casos de usos apresentados na especificação dentro da visão de negócios obtém-se três grandes fronteiras de serviços, casos de usos e consequentemente de entidades informacionais que serão persistidas na base de dados: agendamentos, gerenciamento financeiro e gerenciamento de recursos da empresa (dados da empresa, funcionários, serviços e clientes).

Após essa análise decidiu-se então organizar a base de dados em três grandes grupos de entidades de dados:

- **Appointments:** Grupo de entidades relacionadas aos agendamentos e agendas diárias dos funcionários, como agendamentos, horários de trabalho e bloqueios de agenda.
- **Financial management:** Grupo de entidades relacionadas ao gerenciamento financeiro da empresa, como entradas de caixa, despesas, contas a pagar e contas a receber.
- **Customers:** Grupo de entidades relacionadas ao gerenciamento de recursos da empresa como dados da empresa, funcionários, serviços e clientes da empresa.

Estes três grupos foram consolidados em três schemas do banco de dados do projeto, facilitando portanto a potencial completa isolamento destes em bancos de dados distintos. Tais schemas, juntamente com suas entidades, são apresentados no diagrama a seguir.

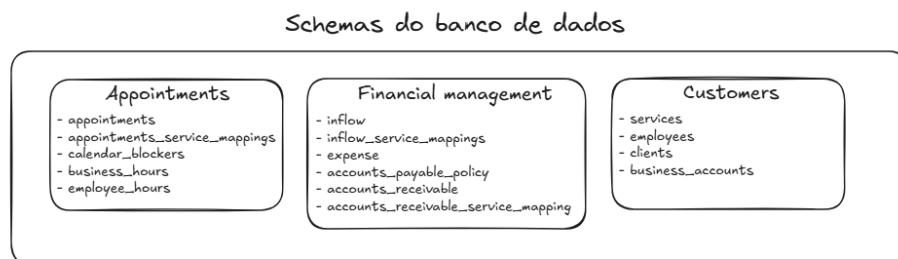


Figura 6 – Schemas do banco de dados

A decisão de organizar estas entidades em três schemas distintos do banco de dados fundamenta-se em princípios consolidados de engenharia de software para sistemas distribuídos, particularmente os conceitos de separação de responsabilidades (*Separation*

of Concerns) e contextos delimitados (*Bounded Contexts*) propostos por *Domain-Driven Design* (EVANS, 2003). Cada schema agrupa entidades que possuem alta coesão semântica e ciclo de vida independente, minimizando o acoplamento entre diferentes áreas funcionais do sistema e facilitando a compreensão e manutenção. Esta organização prepara a base de dados para uma potencial evolução arquitetural em direção a microserviços, permitindo que cada schema seja facilmente migrado para um banco de dados independente, suportando o princípio de *database per service* e oferecendo flexibilidade para escalar cada componente de forma independente conforme demandas específicas.

Adicionalmente, a separação facilita otimizações específicas para cada domínio, implementação de políticas de controle de acesso granulares e isolamento de transações, estabelecendo fundações para uma arquitetura flexível, evolutiva e alinhada com as melhores práticas para desenvolvimento de sistemas distribuídos escaláveis e manuteníveis.

A seguir, exploraremos as tabelas pertencentes a cada schema, aprofundando-nos em como os dados são persistidos na base de dados do sistema. O diagrama abaixo corresponde às tabelas pertencentes ao schema de *Customers*, em sequência detalha-se as colunas e responsabilidades de cada tabela.

customers.customer_account	
id	character varying NN
business_name	character varying NN
display_name	character varying NN
business_address	character varying NN

customers.employee	
id	character varying NN
customer_account_id	character varying NN
first_name	character varying NN
last_name	character varying NN
mail	character varying NN
phone_number	character varying NN
display_name	character varying NN

customers.service	
id	character varying NN
customer_account_id	character varying NN
name	character varying NN
amount	character varying NN
duration	integer NN

customers.employee_service_mappings	
id	character varying NN
employee_id	character varying NN
customer_account_id	character varying NN
service_id	character varying NN

Figura 7 – Tabelas do schema de customers

- **customer_account:** tabela que representa a empresa. É composta pelos campos: **id** (identificador único do funcionário), **business_name** (nome da empresa), **display_name** (nome de exibição da empresa), **business_address** (endereço da empresa).
- **employee:** tabela que representa os funcionários da empresa. É composta pelos campos: **id** (identificador único do funcionário), **customer_account_id** (identificador

da empresa), **first_name** (primeiro nome do funcionário), **last_name** (sobrenome do funcionário), **mail** (email do funcionário), **phone_number** (número de telefone do funcionário), **display_name** (nome de exibição do funcionário).

- **service**: tabela que representa os serviços oferecidos pela empresa. É composta pelos campos: **id** (identificador único do serviço), **customer_account_id** (identificador da empresa), **name** (nome do serviço), **amount** (valor do serviço), **duration** (duração do serviço em minutos como número inteiro).

- **employee_service_mappings**: tabela de mapeamento entre funcionários e serviços. É composta pelos campos: **id** (identificador único do mapeamento), **employee_id** (identificador do funcionário), **customer_account_id** (identificador da empresa), **service_id** (identificador do serviço). Permite associar quais serviços cada funcionário está habilitado a realizar.

- **client**: tabela que representa os clientes da empresa. É composta pelos campos: **id** (identificador único do cliente), **display_name** (nome de exibição do cliente), **phone_number** (número de telefone do cliente), **customer_account_id** (identificador da empresa).

O diagrama abaixo corresponde às tabelas pertencentes ao schema de *Appointments*, em sequência detalha-se as colunas e responsabilidades de cada tabela.

appointments.appointments id $\emptyset$ character varying(64) NN - client_id character varying(64) - employee_id character varying(64) NN - start timestamp NN - end timestamp NN - status $\emptyset$ character varying(20) NN - customer_account_id character varying(64) NN	appointments.appointment_financial_mapping id $\emptyset$ character varying(64) NN - appointment_id character varying(64) NN - account_receivable_id character varying(255) - inflow_id character varying(255) - inserted_at timestamp NN - updated_at timestamp NN	appointments.appointment_service_mappings id $\emptyset$ character varying(64) NN - appointment_id character varying(64) NN - service_id character varying(64) NN
appointments.business_hours id $\emptyset$ character varying(64) NN - customer_account_id character varying(64) NN - start time NN - end time NN	appointments.calendar_blockers id $\emptyset$ character varying(64) NN - employee_id character varying(64) NN - start timestamp NN - end timestamp NN - description character varying(255) NN - customer_account_id character varying(64) NN	appointments.employee_hours id $\emptyset$ character varying(64) NN - employee_id character varying(64) NN - start time NN - end time NN

Figura 8 – Tabelas do schema de appointments

- **appointments**: tabela que representa os agendamentos de um funcionário. É composta pelos campos: **id** (identificador único do mapeamento), **customer_account_id** (identificador da empresa), **employee_id** (identificador do funcionário ao qual o agendamento pertence), **client_id** (identificador do cliente associado ao agendamento), **start** e **end** (início e fim do agendamento, respectivamente).

- **appointment_service_mappings**: tabela de mapeamento entre agendamentos e serviços. É composta pelos campos: **id** (identificador único do mapeamento),

`appointment_id` (identificador do agendamento), `service_id` (identificador do serviço associado ao agendamento). Permite que um agendamento possa ter múltiplos serviços associados.

- **business_hours:** tabela que representa o horário de funcionamento da empresa. É composta pelos campos: `id` (identificador único), `customer_account_id` (identificador da empresa), `start` e `end` (horário de início e fim do expediente, respectivamente, armazenados como `TIME`).

- **calendar_blockers:** tabela que representa bloqueios de calendário para funcionários. É composta pelos campos: `id` (identificador único do bloqueio), `employee_id` (identificador do funcionário), `customer_account_id` (identificador da empresa), `start` e `end` (início e fim do bloqueio, respectivamente), `description` (descrição do motivo do bloqueio). Utilizada para marcar períodos em que o funcionário não está disponível para agendamentos.

- **employee_hours:** tabela que representa o horário de trabalho de cada funcionário. É composta pelos campos: `id` (identificador único), `employee_id` (identificador do funcionário), `start` e `end` (horário de início e fim do expediente do funcionário, respectivamente, armazenados como `TIME`).

- **appointment_financial_mapping:** tabela de mapeamento entre agendamentos e informações financeiras. É composta pelos campos: `id` (identificador único), `appointment_id` (identificador do agendamento), `account_receivable_id` (identificador do contas a receber associado), `inflow_id` (identificador da entrada financeira associada). Permite vincular agendamentos com suas respectivas previsões e entradas financeiras.

O diagrama abaixo corresponde às tabelas pertencentes ao schema de *Financial Management*, em sequência detalha-se as colunas e responsabilidades de cada tabela.

financial_management.expense <code>id</code> PK <code>customer_account_id</code> <code>name</code> <code>description</code> <code>total_amount</code> <code>payment_method</code> <code>inserted_at</code> <code>updated_at</code>	financial_management.inflow <code>id</code> PK <code>customer_account_id</code> <code>employee_id</code> <code>total_amount</code> <code>payment_method</code> <code>inserted_at</code> <code>updated_at</code>	financial_management.inflow_service_mapping <code>id</code> PK <code>inflow_id</code> <code>service_id</code> <code>inserted_at</code> <code>updated_at</code>	financial_management.inflow_client_mapping <code>id</code> PK <code>inflow_id</code> <code>client_id</code> <code>inserted_at</code> <code>updated_at</code>
financial_management.accounts_receivable <code>id</code> PK <code>customer_account_id</code> <code>total_amount</code> <code>due_date</code> <code>status</code> <code>payment_method</code> <code>received_at</code> <code>client_id</code> <code>employee_id</code> <code>deleted_at</code> <code>inserted_at</code> <code>updated_at</code>	financial_management.accounts_receivable_service_mapping <code>id</code> PK <code>account_receivable_id</code> <code>service_id</code> <code>deleted_at</code> <code>inserted_at</code> <code>updated_at</code>	financial_management.accounts_payable_policy <code>id</code> PK <code>customer_account_id</code> <code>name</code> <code>default_amount</code> <code>recurrence</code> <code>first_date</code> <code>deleted_at</code> <code>inserted_at</code> <code>updated_at</code>	financial_management.accounts_payable <code>id</code> PK <code>customer_account_id</code> <code>policy_id</code> <code>name</code> <code>total_amount</code> <code>recurrence</code> <code>due_date</code> <code>status</code> <code>payment_method</code> <code>paid_at</code> <code>deleted_at</code> <code>inserted_at</code> <code>updated_at</code>

Figura 9 – Tabelas do schema de financial management

- **inflow:** tabela que representa as entradas financeiras (receitas recebidas). É composta pelos campos: `id` (identificador único da entrada), `customer_account_id` (identificador da empresa), `employee_id` (identificador do funcionário responsável pela entrada), `total_amount` (valor total da entrada), `payment_method` (método de pagamento utilizado), `inserted_at` e `updated_at` (timestamps de criação e atualização do registro).
- **inflow_service_mapping:** tabela de mapeamento entre entradas financeiras e serviços prestados. É composta pelos campos: `id` (identificador único do mapeamento), `inflow_id` (identificador da entrada financeira), `service_id` (identificador do serviço associado), `inserted_at` e `updated_at` (timestamps de criação e atualização).
- **inflow_client_mapping:** tabela de mapeamento entre entradas financeiras e clientes. É composta pelos campos: `id` (identificador único do mapeamento), `inflow_id` (identificador da entrada financeira), `client_id` (identificador do cliente), `inserted_at` e `updated_at` (timestamps de criação e atualização). Permite associar entradas financeiras aos clientes que as geraram.
- **expense:** tabela que representa as despesas da empresa. É composta pelos campos: `id` (identificador único da despesa), `customer_account_id` (identificador da empresa), `name` (nome da despesa), `description` (descrição opcional da despesa), `total_amount` (valor total da despesa), `payment_method` (método de pagamento utilizado), `inserted_at` e `updated_at` (timestamps de criação e atualização).
- **accounts_payable_policy:** tabela que representa políticas/modelos de contas a pagar recorrentes. É composta pelos campos: `id` (identificador único da política), `customer_account_id` (identificador da empresa), `name` (nome da política), `default_amount` (valor padrão), `recurrence` (frequência de recorrência), `first_date` (data da primeira ocorrência), `deleted_at` (timestamp de exclusão lógica), `inserted_at` e `updated_at` (timestamps de criação e atualização). Serve como template para geração automática de contas a pagar.
- **accounts_payable:** tabela que representa as contas a pagar (despesas futuras). É composta pelos campos: `id` (identificador único), `customer_account_id` (identificador da empresa), `policy_id` (identificador da política que gerou esta conta), `name` (nome da conta), `total_amount` (valor total), `recurrence` (frequência de recorrência), `due_date` (data de vencimento), `status` (status da conta), `payment_method` (método de pagamento), `paid_at` (timestamp do pagamento, se já foi paga), `deleted_at` (timestamp de exclusão lógica), `inserted_at` e `updated_at` (timestamps de criação e atualização).
- **accounts_receivable:** tabela que representa as contas a receber (receitas futuras). É composta pelos campos: `id` (identificador único), `customer_account_id` (identifica-

dor da empresa), `total_amount` (valor total), `due_date` (data de vencimento), `status` (status da conta), `payment_method` (método de pagamento), `received_at` (timestamp do recebimento, se já foi recebida), `client_id` (identificador do cliente devedor), `employee_id` (identificador do funcionário responsável), `deleted_at` (timestamp de exclusão lógica), `inserted_at` e `updated_at` (timestamps de criação e atualização).

- **`accounts_receivable_service_mapping`**: tabela de mapeamento entre contas a receber e serviços prestados. É composta pelos campos: `id` (identificador único do mapeamento), `account_receivable_id` (identificador da conta a receber), `service_id` (identificador do serviço associado), `deleted_at` (timestamp de exclusão lógica), `inserted_at` e `updated_at` (timestamps de criação e atualização). Permite associar múltiplos serviços a uma conta a receber.

A partir dessa separação lógica dos dados gerenciados na plataforma, é possível expandir essa definição para outros aspectos da arquitetura, conforme será destacado nos capítulos posteriores.

4.2 Visão computacional

A arquitetura proposta fundamenta-se na decomposição de microsserviços orientada por responsabilidades de negócio, alinhando-se diretamente com a visão de business drivers da nossa adaptação do RM-ODP. Cada microsserviço encapsula não apenas a lógica de negócio específica de seu domínio, mas também mantém interfaces bem definidas para integração entre os diferentes módulos de negócio presentes na versão construída da plataforma. Esta estratégia permite que modificações em um domínio específico sejam isoladas, reduzindo o acoplamento entre componentes e facilitando a manutenção evolutiva do sistema.

A extensibilidade constitui um atributo arquitetural fundamental da solução proposta, viabilizada pela natureza modular dos microsserviços e pela aderência aos princípios de separação de responsabilidades. A arquitetura foi concebida para atender a diversos tipos de pequenos negócios através da adição incremental de novos módulos, sem necessidade de alterações substanciais na infraestrutura existente. Esta capacidade de extensão horizontal manifesta-se tanto na incorporação de novos domínios de negócio quanto na especialização de funcionalidades existentes para atender segmentos específicos do mercado.

O núcleo da infraestrutura fundamenta-se na comunicação inter-serviços através do protocolo gRPC (Google Remote Procedure Call) ([gRPC Authors, 2024](#)), uma escolha arquitetural que reflete os requisitos não funcionais de performance e confiabilidade identificados durante a especificação do projeto. O gRPC oferece vantagens significativas em relação aos protocolos tradicionais baseados em REST/HTTP, incluindo serialização binária mais eficiente através do Protocol Buffers, suporte nativo a streaming bidirecional e geração

automática de clientes em múltiplas linguagens. Esta abordagem garante comunicação de baixa latência entre os microsserviços, aspecto crítico para a experiência do usuário.

A interface com a camada de apresentação é mediada por um API Gateway que atua como ponto de entrada unificado para todas as requisições HTTP provenientes das aplicações cliente. Este componente arquitetural desempenha papel fundamental na tradução de protocolos, convertendo requisições HTTP/REST em chamadas gRPC para os microsserviços internos e posteriormente transformando as respostas gRPC de volta para o formato HTTP/JSON esperado pelas aplicações web e mobile. Dessa forma, resume-se a atuação do componente como um proxy reverso e agregador de serviços, implementando as políticas de comunicação que atendem aos requisitos de segurança e baixa latência definidos anteriormente em 3.

A escolha desta arquitetura híbrida permite que a plataforma capitalize os benefícios de ambos os paradigmas de comunicação. Internamente, os microsserviços beneficiam-se da eficiência e robustez do gRPC, enquanto externamente, a plataforma mantém compatibilidade com o ecossistema web padrão através de interfaces HTTP/REST.

Do ponto de vista da escalabilidade, a arquitetura obtida demonstra características favoráveis através da decomposição em microsserviços autônomos e da comunicação assíncrona suportada pelo gRPC. Cada microsserviço pode ser escalado independentemente com base em suas demandas. O API Gateway, por sua vez, pode ser implementado em configuração de alta disponibilidade através de load balancers, garantindo que não se torne um ponto único de falha na arquitetura. A arquitetura da aplicação portanto se dá conforme a imagem abaixo.

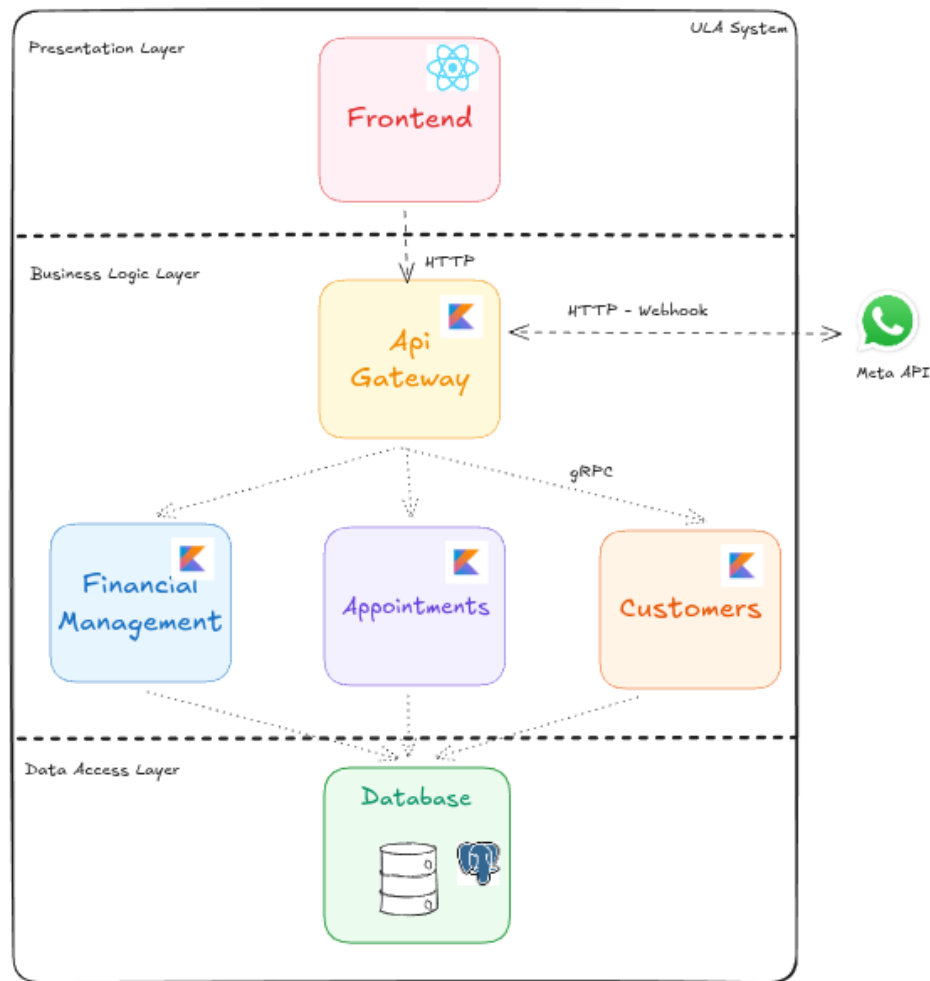


Figura 10 – Arquitetura completa da aplicação

A arquitetura implementada representa compatibilidade tanto com os requisitos funcionais específicos dos pequenos negócios quanto com os requisitos não funcionais de performance, escalabilidade e manutenibilidade.

4.3 Visão tecnológica

Nesta seção, destaca-se a implementação da plataforma em termos das tecnologias usadas, das metodologias aplicadas e dos resultados obtidos nos repositórios de código. Na camada de plataforma e backend, as tecnologias (PostgreSQL e Kotlin) ([PostgreSQL Global Development Group, 2024](#)) ([JetBrains s.r.o., 2024](#)) foram selecionadas devido ao seu amplo uso em cenários de aplicações hospedadas em nuvem no mercado. Já na camada de apresentação utiliza-se o arcabouço *React* ([Meta Platforms, Inc., 2024](#)), baseado na linguagem *Typescript* ([Microsoft Corporation, 2024](#)), devido à sua ampla utilização no mercado e suporte de manutenção fornecido por grandes empresas. Para a etapa de implementação do projeto, utilizou-se uma metodologia ágil análoga ao scrum, alinhada

aos processos de ciclo de vida de software definidos pela ISO/IEC 12207 (ISO/IEC, 2017). Os membros do grupo se reuniram de forma estruturada para garantir o andamento do projeto, realizando duas reuniões por semana: uma no início, para definir o que deveria ser feito e outra no final para acompanhamento do progresso.

Para a organização do código desenvolvido, escolheu-se a abordagem monorepo (GLEISON; RICARDO; VALENTE, 2018), amplamente divulgada pela Google. Essa abordagem define que todo o código deve ficar em um único sistema de versionamento. Tal metodologia garante que todos os envolvidos tenham acesso a todos os módulos e também facilita o gerenciamento de dependências dentro da aplicação, pois é possível garantir que todos os módulos utilizem as mesmas bibliotecas, evitando erros de versionamento e compatibilidade.

Ademais, a organização do projeto em módulos dentro de um mesmo repositório permite que o código reflita a arquitetura idealizada de maneira mais estruturada. Dessa forma, a organização do projeto no GitHub se dá como:

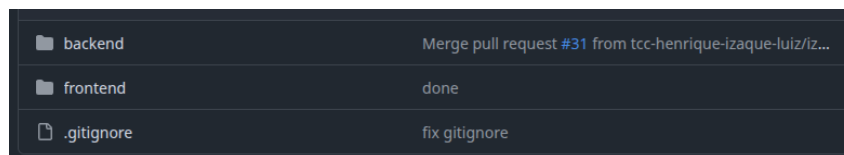


Figura 11 – Organização de pastas geral

Para exemplificação da centralização de dependências, toma-se como exemplo o módulo de backend, contando com um submódulo library, de onde outras partes do código podem utilizar funções comuns:

api_gateway	Merge branch 'main' into izaque/agendamento-finance
appointments	-
customers	fix and remove prints
financial_management	backend working
gradle	backend working
library	done
proto-shared	Merge branch 'main' into izaque/agendamento-finance
.env.local	Env var wpp
.env.prod	Env var wpp
Makefile	backend working
README.md	backend working
build.gradle.kts	tested api gateway
deploy.sh	done
docker-compose.local.yml	done
docker-compose.prod.yml	done
docker-compose.yml	Env var wpp
gradle.properties	backend minimal structure
gradlew	backend minimal structure
gradlew.bat	backend minimal structure
settings.gradle.kts	done
update.sh	done

Figura 12 – Organização de pastas backend

Em código, a arquitetura da plataforma é contemplada pela separação em módulos, cada um com compilação e construção independente, refletindo a arquitetura de microsserviços definida em 4.2. Cada um dos microsserviços é implementado utilizando a linguagem Kotlin (JetBrains s.r.o., 2024) e seus arcabouços para desenvolvimento de aplicações Web. Destaca-se os microsserviços em código na imagem abaixo:

api_gateway	done	yesterday
appointments	done	yesterday
customers	done	yesterday
financial_management	be done	3 weeks ago

Figura 13 – Microsserviços da plataforma implementados em código

Expandindo sobre a arquitetura que foi implementada, destaca-se que cada microsserviço correspondente a um aspecto da lógica de negócios foi devidamente separado em camadas, conforme mostrado na imagem 10. De modo geral, cada um dos microsserviços é composto pelas seguintes camadas:

- *app* - *entrypoint* da aplicação, função que deve ser executada para a inicialização do sistema.
- *data* - componentes responsáveis pela integração com banco de dados.
- *grpc* - componentes responsáveis pela compatibilidade com o protocolo gRPC.

- *model* - definição das classes que representam a lógica de negócios.
- *services* - camada que utiliza os elementos de *data* e de *model* para execução de rotinas referentes à lógica de negócios.

A organização em código de cada serviço pode ser explorada nas imagens abaixo:

app	add ignore auto message
constants	updated with main
data	add ignore auto message
grpc	fix and remove prints
model	add ignore auto message
services	fix and remove prints

Figura 14 – Implementação em código do microserviço customer

..		
app	removing zone id + comments	2 months ago
constants	removing zone id + comments	2 months ago
data	backend tested	last month
grpc	backend tested	last month
model	backend tested	last month
services	backend tested	last month

Figura 15 – Implementação em código do microserviço de agendamentos

..		
app	finished grpc layer	last month
data	tested pay api gateway	last month
grpc	tested pay api gateway	last month
models	finished grpc layer	last month
services	tested pay api gateway	last month

Figura 16 – Implementação em código do microserviço de gestão financeira

É válido mencionar que as camadas de persistência presentes na solução e destacadas na arquitetura da imagem correspondem à separação lógica em schemas explicada em 4.1, como parte da premissa de desacoplamento entre os módulos.

Conforme mencionado anteriormente, para a compatibilidade entre gRPC e HTTP (utilizado pela maioria dos navegadores para carregamento de páginas na Web), elabora-se um componente denominado *API Gateway*, que conta com os seguintes módulos:

- *app* - *entrypoint* da aplicação, função que deve ser executada para a inicialização do sistema.
- *clients/grpc* - componentes responsáveis pela comunicação com os microserviços.
- *controllers* - controladores HTTP que recebem as requisições do frontend.

- *dtos* - classes que definem como as classes de negócio são transferidas através da rede.
- *resolvers* - elementos utilizados pelos *controllers* para mapear cada rota a um microsserviço específico.

A implementação da arquitetura do API Gateway em código é:

..		
app	migration	3 months ago
clients/gpsc	fix customer name	3 weeks ago
config	implemented singleton and tested	4 months ago
constants	fix customer name	3 weeks ago
controllers	Fix employee_id to id	3 weeks ago
dtos	complete ui	3 weeks ago
resolvers	fix customer name	3 weeks ago

Figura 17 – Implementação em código do API Gateway

Da mesma forma, o frontend foi implementado utilizando-se o arcabouço *React* (Meta Platforms, Inc., 2024), baseado na linguagem *Typescript* (Microsoft Corporation, 2024). Este módulo representa a camada de apresentação e também é modularizado através de uma arquitetura em camadas, sendo elas:

- *api* - componentes responsáveis pela integração HTTP com o API Gateway.
- *components* - componentes visuais exibidos na tela.
- *hooks* - componentes responsáveis pela lógica nas telas.
- *pages* - páginas a serem exibidas em cada rota da aplicação.
- *types* - definição da tipagem das classes de negócio da aplicação.

Em código, estes se refletem em:

..		
apis	Merge pull request #33 from tcc-henrique-izaque-luiz/registerBack	3 weeks ago
components	Fix text	2 days ago
config	complete ui	3 weeks ago
constants	Merge remote-tracking branch 'origin/main' into servicesFront	3 weeks ago
hooks	complete ui	3 weeks ago
images	Full back employee	last month
pages	complete ui	3 weeks ago
themes	styling	last month
types	Merge remote-tracking branch 'origin/main' into servicesFront	3 weeks ago
utils	resolved comment	last month
App.test.tsx	front + tailwind and without vite	4 months ago
App.tsx	Merge remote-tracking branch 'origin/main' into servicesFront	3 weeks ago

Figura 18 – Implementação em código do frontend

Aqui evidenciamos a implementação tecnológica das definições arquiteturais feitas previamente no trabalho, destacando a arquitetura de serviços de acordo com a visão

computacional. A seguir, detalharemos a visão de engenharia, com ênfase na infraestrutura criada para suportar o projeto.

4.4 Visão de Engenharia

A infraestrutura definida visa suportar a arquitetura proposta em microsserviços, visando garantir os requisitos não funcionais definidos em 3, durante a definição dos mesmos. A arquitetura infraestrutural do sistema foi projetada seguindo os princípios de microsserviços, containerização e computação em nuvem. A solução é hospedada integralmente na Amazon Web Services (AWS) ([Amazon Web Services, 2024](#)), aproveitando serviços gerenciados para garantir escalabilidade, disponibilidade e facilidade de manutenção. Toda a infraestrutura é isolada em uma Virtual Private Cloud (VPC), proporcionando segurança de rede e controle granular sobre o tráfego de dados.

A camada de apresentação é servida através do serviço de armazenamento em nuvem fornecido pelo Amazon S3 (Simple Storage Service), neste caso especificamente configurado para hospedagem de sites estáticos. Esta abordagem oferece alta disponibilidade, baixa latência e custos reduzidos para servir conteúdo do site. O S3 atua como origem para os assets da aplicação web, incluindo HTML, CSS, JavaScript e recursos multimídia. A comunicação entre o frontend e a plataforma ULA ocorre via protocolo HTTP/HTTPS, direcionando as requisições para o API Gateway, que atua como ponto único de entrada para todos os serviços das camadas da lógica de negócio.

Os microsserviços criados e evidenciados na arquitetura do sistema mostrada em 10 são servidos em máquinas do serviço Amazon EC2 (Elastic Compute Cloud), serviço que fornece capacidade de computação segura e redimensionável na nuvem através de máquinas virtuais. Cada serviço é executado em seu próprio ambiente isolado, utilizando a ferramenta de containerização Docker ([Docker Inc., 2024a](#)), sendo executadas nas máquinas fornecidas pelo serviço em nuvem. Os containers são orquestrados através do Docker Compose ([Docker Inc., 2024b](#)), que define a configuração de rede, volumes, variáveis de ambiente e dependências entre serviços.

Para a camada de persistência utiliza-se o Amazon RDS (Relational Database Service) com PostgreSQL ([PostgreSQL Global Development Group, 2024](#)) como sistema gerenciador de banco de dados. A escolha por um banco de dados relacional único simplifica a gestão de transações distribuídas e consultas que envolvem múltiplos domínios, conforme foi apresentado anteriormente na separação de schemas.

Os serviços utilizados para criar a infraestrutura podem ser sintetizados através da arquitetura abaixo:

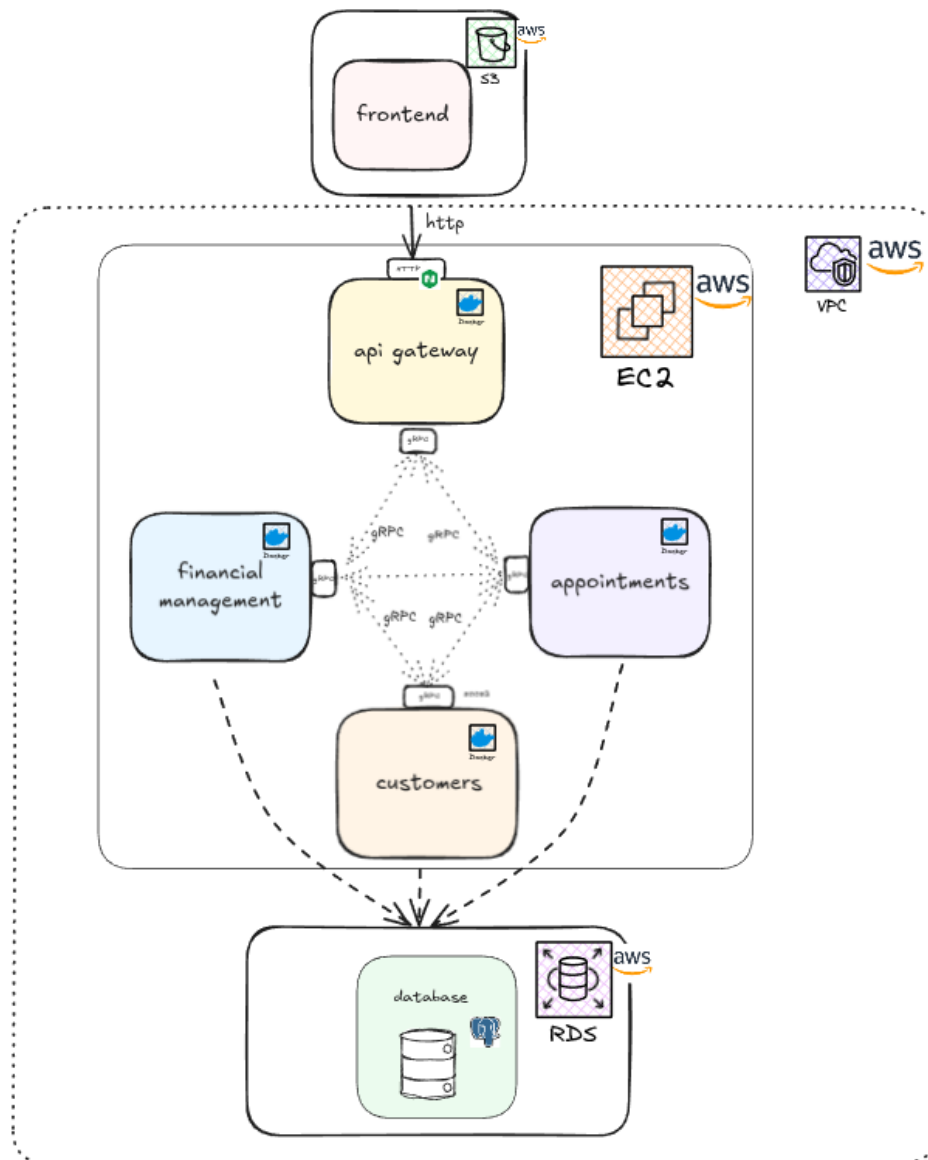


Figura 19 – Arquitetura da visão de engenharia

4.5 Telas e experiência do usuário

O início da plataforma bem como qualquer negócio se dá com serviços, funcionários e clientes e para tal foram implementados fluxos para realizar o gerenciamento desses com foco usabilidade propondo maior velocidade em registros e, no futuro, permite melhor organização e abstração de processos do ponto de vista do gerente da empresa. Para o Projeto de Formatura foram abstraídas algumas funcionalidades gerais como o próprio cadastro da empresa, que foi fixada para criação do exemplo a ser apresentado, outras funcionalidades deixadas de lado para o objeto de estudo e consequências serão abordadas no Capítulo 5 quando aborda a continuidade do trabalho.

Quanto às funcionalidades do gerenciamento de funcionários, representadas na

página e respectivos modais das imagens a seguir, é possível realizar comandos básicos de cadastrar, editar e excluir registros de funcionários mas também permite realizar a associação de serviços a funcionários, fundamental para realizar corretamente o agendamento, tudo realizado através de modais intuitivos e pragmáticos pensados em maior usabilidade visando facilitar as tarefas de um gerente de pequeno negócio, que muitas vezes é o dono e funcionário de seu próprio negócio, ou seja, possui muitas tarefas e responsabilidades diárias, não havendo tempo para processos longos.

Ressalta-se que nas imagens selecionadas foram utilizados dados aleatórios e totalmente fictícios, gerados para a documentação, nenhum dado apresentado tem relação com qualquer pessoa ou empresa.

Gerenciamento de funcionários

Registrar novo funcionário

NOME	TELEFONE	EMAIL	SERVIÇOS	
Aline Cardoso	11996678855	aline.cardoso@example.com	Plano Básico, Auditoria Completa e outros 2	 
Camila Rocha	11995553321	camila.rocha@example.com	Nenhum serviço	 
Felipe Ramos	11994447711	felipe.ramos@example.com	Manutenção Premium	 
Henrique Lopes	11997771212	henrique.lopes@example.com	Nenhum serviço	 
Larissa Cruz	11992224488	larissa.cruz@example.com	Nenhum serviço	 
Patricia Almeida	11999032178	patricia.almeida@example.com	Auditoria Completa	 
Rafael Gomes	11983002245	rafael.gomes@example.com	Nenhum serviço	 

< Anterior

1

2

Próxima >

Figura 20 – Tela de gerenciamento de funcionários



The image shows a mobile application interface with a modal for registering a new employee. The modal is titled "Cadastrar Novo Funcionário" and has a close button (X) in the top right corner. It contains four input fields: "Primeiro Nome", "Último Nome", "Email", and "Telefone". The "Telefone" field has a placeholder text: "Apenas números, por exemplo: 11912341234". At the bottom of the modal is a button labeled "Cadastrar Funcionário". The background shows a list of employees with columns for phone number, email, and plan name.

Telefone	Email	Plano
11996678855	aline.cardoso@example.com	Plano Básico
11995		m s
11994		enç
11997		m s
11992		m s
11999		ria C
11983		m s

Figura 21 – Modal para cadastrar um novo funcionário

Em casos de mudanças de dados, basta acessar o modal de atualização apertando o botão com ícone de lápis e atualizar aquilo que for necessário.

A modal window titled "Atualizar dados do Funcionário" with a close button (X) in the top right corner. It contains four input fields: "Primeiro Nome" with the value "Aline", "Último Nome" with the value "Cardoso", "Email" with the value "aline.cardoso@example.com", and "Telefone" with the value "11996678855". At the bottom, there are two buttons: a gray "Cancelar" button and an orange "Atualizar" button.

Atualizar dados do Funcionário

Primeiro Nome: Aline

Último Nome: Cardoso

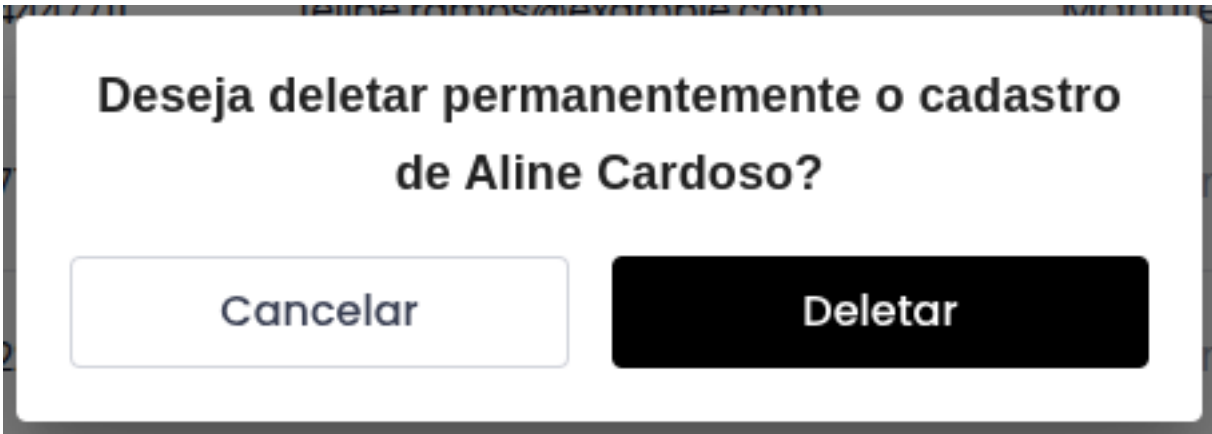
Email: aline.cardoso@example.com

Telefone: 11996678855

Cancelar Atualizar

Figura 22 – Modal para atualizar dados de um funcionário

Quando um funcionário for desligado da empresa, com um simples fluxo é possível remover sua referência, assim não será mais possível realizar agendamentos para esses.

A modal window with the title "Deseja deletar permanentemente o cadastro de Aline Cardoso?". It features two buttons at the bottom: a white "Cancelar" button and a black "Deletar" button.

Deseja deletar permanentemente o cadastro de Aline Cardoso?

Cancelar Deletar

Figura 23 – Modal para deletar um funcionário

Para controlar quais funcionários que devem realizar determinados serviços considerando habilidades e competências individuais cria-se uma referência dos serviços associando a cada funcionário uma lista de serviços que ele pode atender.



Aline Cardoso

Serviços

Selecione um serviço

Auditoria Completa x Plano Básico x

Plano Plus x Plano Starter x

Confirmar associação

Figura 24 – Modal para associar serviços a funcionários

Quanto às funcionalidades do gerenciamento de serviços, seguindo fluxo parecido ao do gerenciamento de funcionários e baseado nos mesmos princípios, é possível cadastrar, editar e excluir serviços, tudo evidenciado nas imagens a seguir.

Gerenciamento de serviços			Cadastrar Serviço
NOME	VALOR	DURAÇÃO	
Auditoria Completa	R\$1200.00	120 min	 
Consultoria Básica	R\$150.00	7 min	 
Consultoria Estratégica	R\$2500.00	30 min	 
Manutenção Premium	R\$399.00	90 min	 
Plano Básico	R\$49.90	15 min	 
Plano Empresarial	R\$899.90	180 min	 
Plano Plus	R\$129.90	60 min	 
 Anterior			1 2 Próxima 

Figura 25 – Tela de gerenciamento de serviços



The image shows a modal window titled "Cadastrar Novo Serviço" (Register New Service) with a close button (X) in the top right corner. The modal contains three input fields and a submit button. The first field is labeled "Nome" (Name) and has a placeholder "Ex: Corte de cabelo masculino" (Ex: Men's haircut). The second field is labeled "Valor" (Value) and has a placeholder "Ex: R\$ 35,00" (Ex: R\$ 35.00). The third field is labeled "Duração (em minutos)" (Duration in minutes) and has a placeholder "0". At the bottom of the modal is a large button labeled "Cadastrar Serviço" (Register Service).

Cadastrar Novo Serviço ✕

Nome

Ex: Corte de cabelo masculino

Valor

Ex: R\$ 35,00

Duração (em minutos)

0

Cadastrar Serviço

Figura 26 – Modal para cadastrar um novo serviço

Nenhum serviço é imutável, para tanto permite-se facilmente alterar o nome do serviço, que para cenário de pequenos negócios é considerado também como descrição, valor e duração. Um modo simples de controlar os preços, permitindo até possíveis promoções. Além disso, caso deixe de oferecer determinado serviço pode também deletá-lo, tornando-o indisponível para futuros agendamentos.

A modal window titled "Atualizar dados do Serviço" with a close button (X) in the top right corner. It contains three input fields: "Nome do Serviço" with the value "Auditoria Completa", "Valor" with the value "1200.00", and "Duração (minutos)" with the value "120". At the bottom, there are two buttons: "Cancelar" (gray) and "Atualizar" (orange).

Atualizar dados do Serviço X

Nome do Serviço

Auditoria Completa

Valor

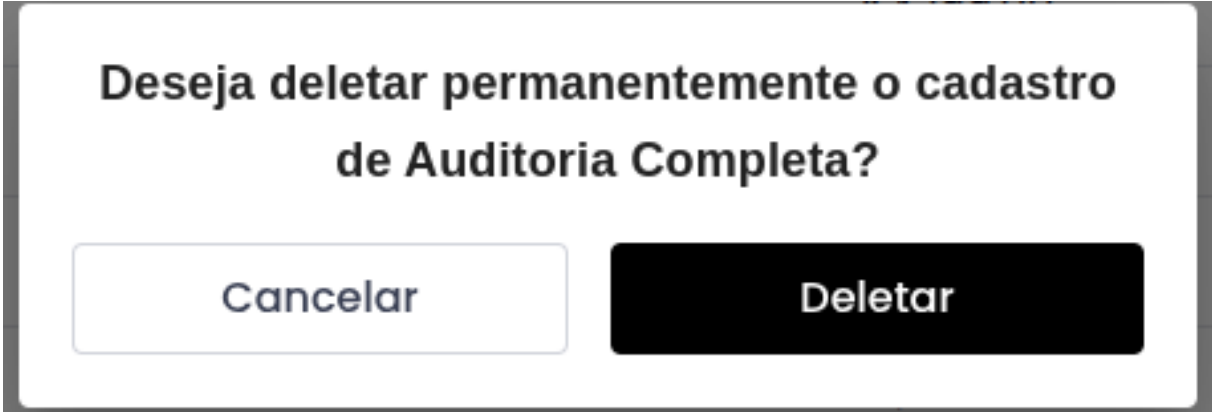
1200.00

Duração (minutos)

120

Cancelar Atualizar

Figura 27 – Modal para atualizar dados de um serviço

A modal window with the text "Deseja deletar permanentemente o cadastro de Auditoria Completa?". At the bottom, there are two buttons: "Cancelar" (white with gray border) and "Deletar" (black with white text).

Deseja deletar permanentemente o cadastro de Auditoria Completa?

Cancelar Deletar

Figura 28 – Modal para deletar um serviço

Em relação aos clientes há um fluxo diferente, a plataforma inclui uma aplicação que com auxílio da API do aplicativo de mensagens WhatsApp realiza o cadastro dos clientes automaticamente com informações públicas disponíveis pela API, nome e número de telefone do usuário, não sendo necessário que o cliente apresente outros dados para o cadastro e assim não há necessidade de alocar o tempo de um funcionário para cadastrar clientes, além de que sempre teremos os dados dos clientes na hora do agendamento.

Devido ao fato de que o nome pode ser personalizado nesse aplicativo e a automação não fazer distinção disso será possível alterar o nome do cliente pela plataforma para ter

certeza sempre de qual cliente se trata em agendamentos futuros.

Gerenciamento de clientes		
NOME	TELEFONE	
Ana Pereira	11991824437	
Beatriz Nunes	11997231155	
Carlos Souza	11987452210	
Fernanda Costa	11998332291	
Gustavo Lima	11990123344	
João Mendes	11995344678	
Juliana Castro	11997556677	
< Anterior12Próxima >		

Figura 29 – Tela de gerenciamento de clientes

Atualizar nome do Cliente

Nome

Ana Pereira

Cancelar

Atualizar

Figura 30 – Modal para atualizar o nome de um cliente

Como mostrado nas imagens abaixo o cliente envia uma mensagem que, ao conter palavras chaves configuradas para iniciar um agendamento, no momento em que está sendo processada busca o registro do cliente através do número de telefone dele, se não encontrar realiza seu cadastro, então gera-se uma mensagem personalizada com o nome cadastrado e a automação a envia para o cliente com o link de agendamento que será melhor explicado logo depois nesta mesma seção.

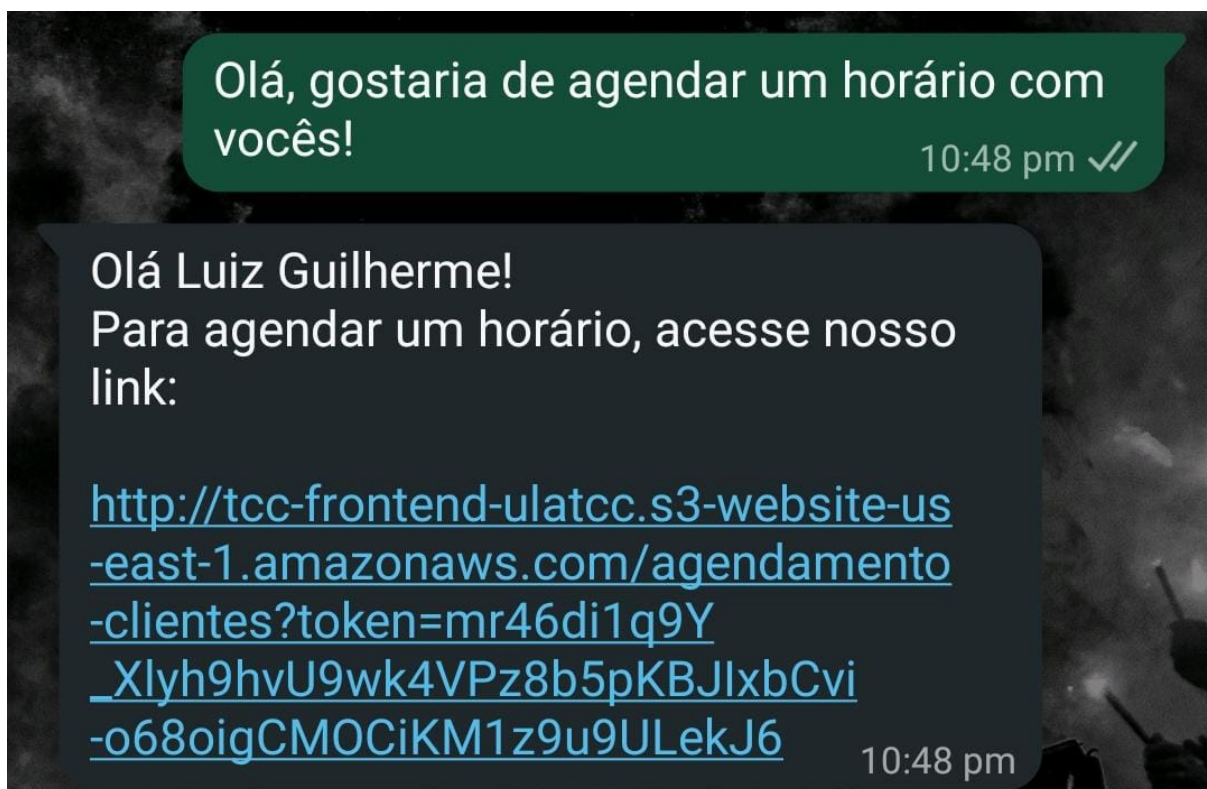


Figura 31 – Exemplo de conversa no WhatsApp

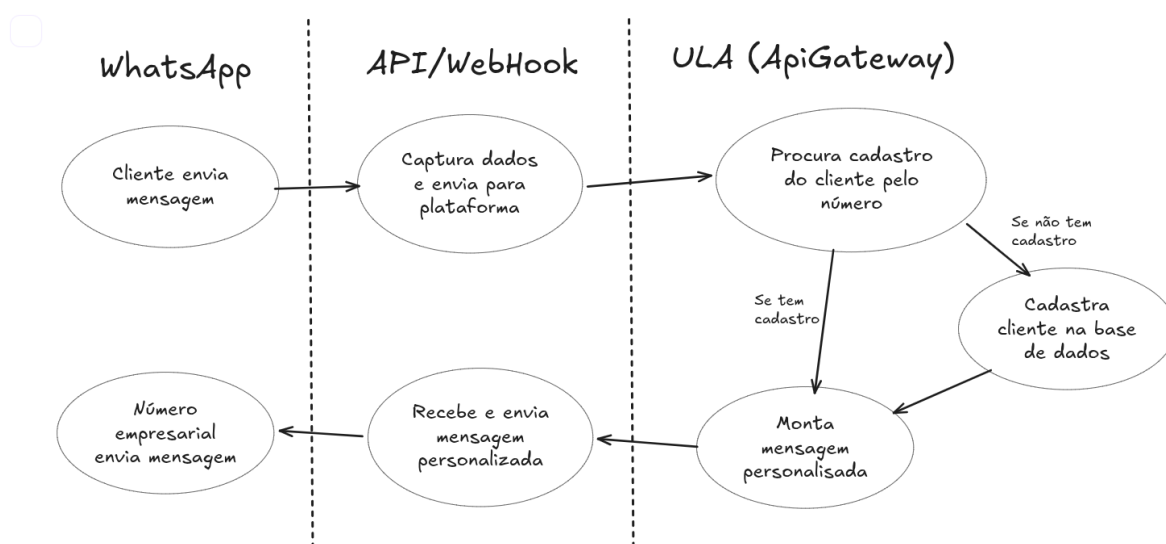


Figura 32 – Fluxo da aplicação com uso do WhatsApp

Preenchendo a camada de negócios temos os principais serviços já implementados quanto ao agendamento, bloqueio de horários e visualização da agenda pelo gerente, funcionário ou o próprio dono de pequeno negócio.

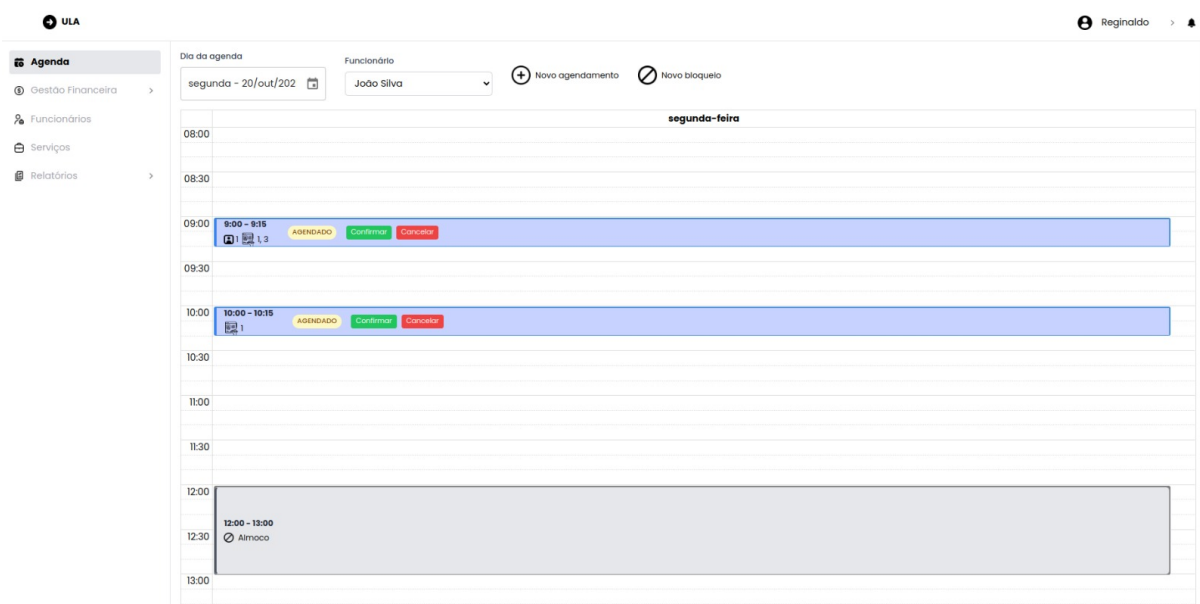


Figura 33 – Visualização completa da agenda

Bloquear Calendário

Início

28/09/2025

20:23

Fim

28/09/2025

20:23

Funcionário

Selecione um funcionário

Motivo

Motivo do bloqueio

Bloquear

Figura 34 – Modal para cadastrar um bloqueio na agenda



The image shows a mobile application interface for scheduling a service. At the top, a dark grey header bar contains the word "domingo" in white. Below this, a white modal box titled "Agendar serviço" is centered. The modal contains four sections: "Horário" with a date input showing "28/09/2025" and a time input showing "20:23"; "Funcionário" with a dropdown menu labeled "Selecione um funcionário"; "Serviços" with a dropdown menu labeled "Selecione um serviço"; and "Cliente" with a dropdown menu labeled "Selecione um cliente". At the bottom of the modal is a button labeled "Agendar".

Figura 35 – Modal de agendamento de serviço

Ainda pensando na visão de negócios foram criadas páginas para organizar os gastos e recebimentos que devem auxiliar o dono de pequeno negócio a controlar entradas e saídas de caixas, bem como ter previsibilidade do caixa através das contas a receber.

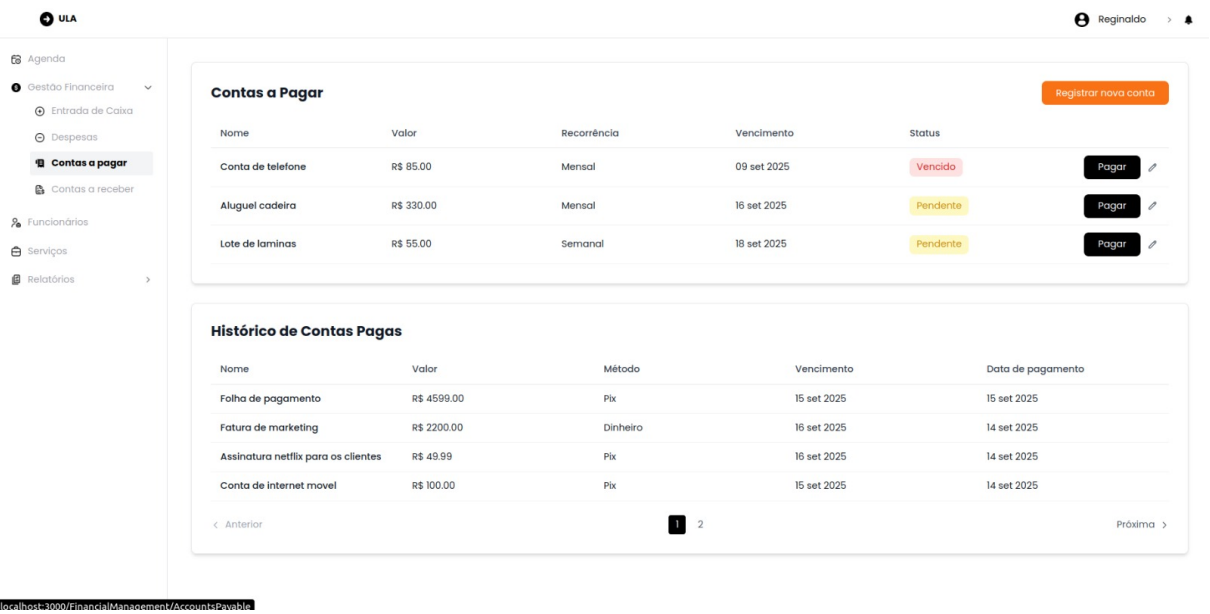


Figura 36 – Componentes principais de contas a pagar

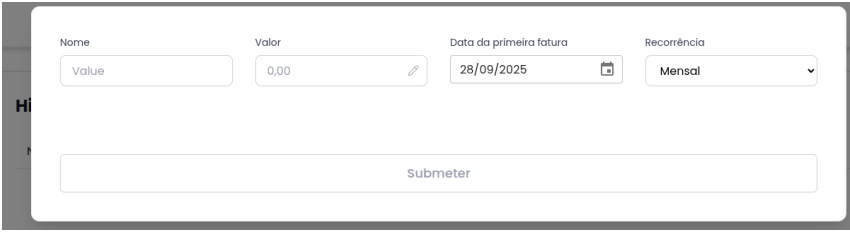


Figura 37 – Modal de registro de nova conta a pagar

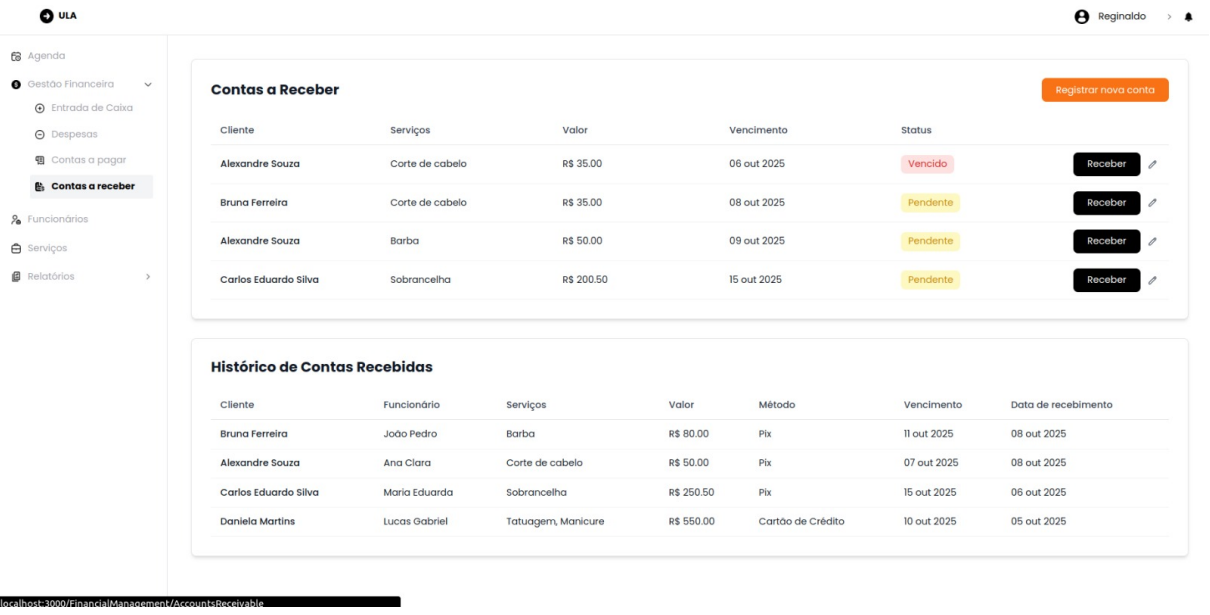


Figura 38 – Componentes principais de contas a receber

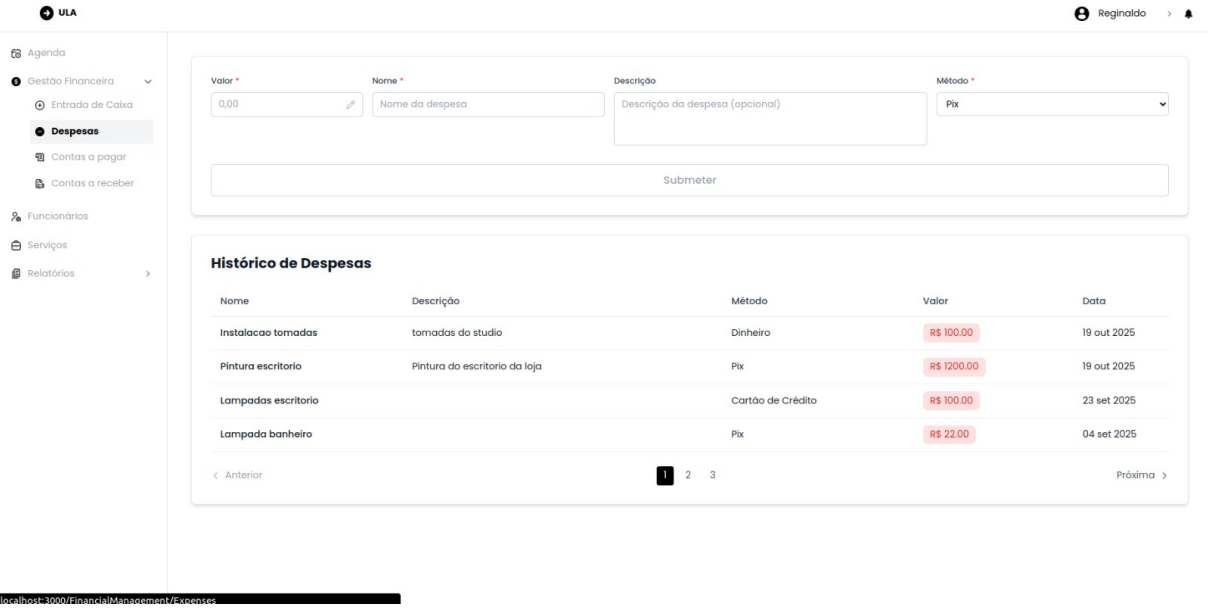


Figura 39 – Principais componentes da página de despesas

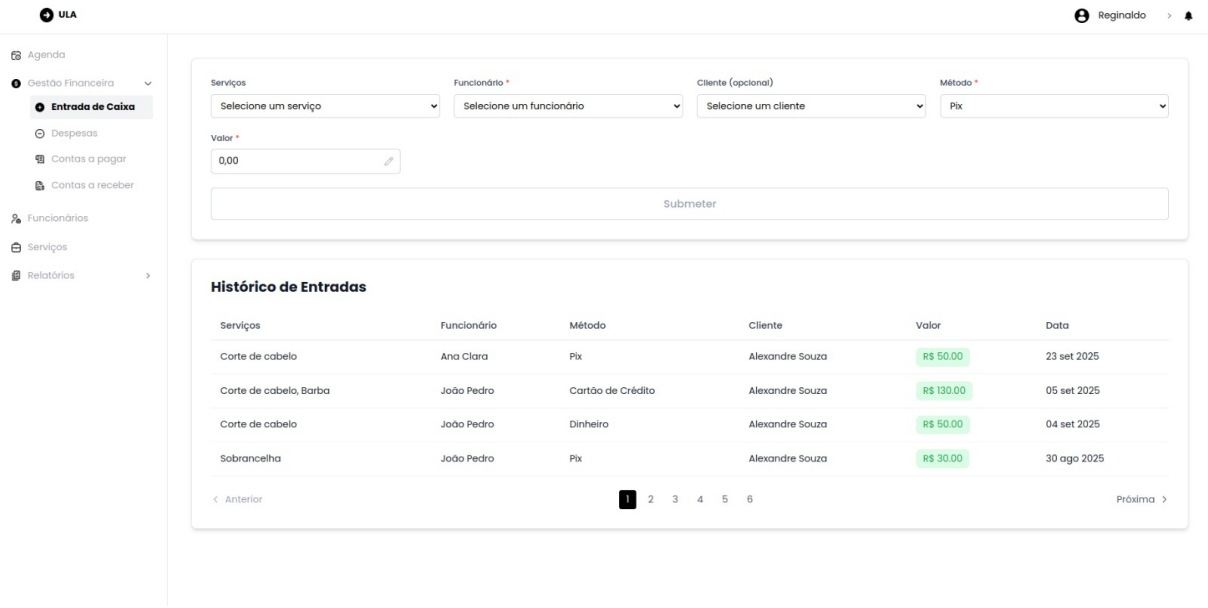


Figura 40 – Principais componentes da página de entradas

A visão geral acerca dos dados financeiros cadastrados na plataforma pode ser consultada no dashboard de saúde financeira. Este componente visa mostrar de forma clara como os diferentes aspectos financeiros afetam na empresa e explica de forma simples como cada uma das variáveis pode ser devidamente manipulada.

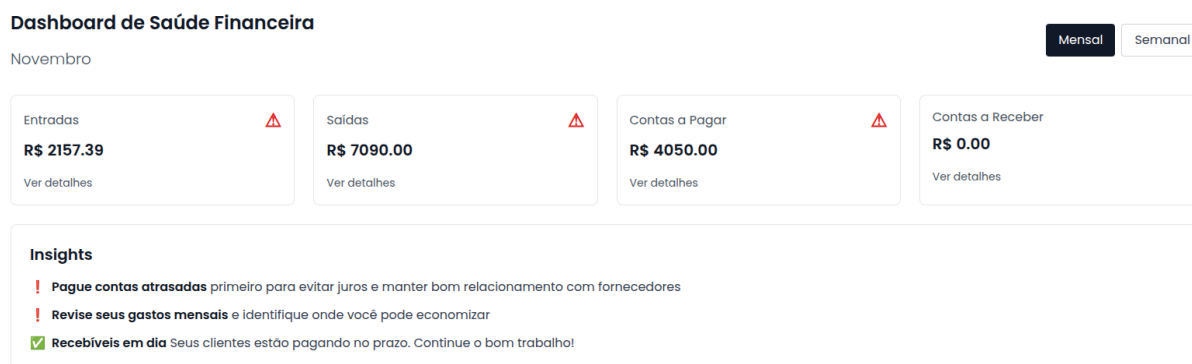


Figura 41 – Dashboard de saúde financeira - controle de granularidade e insights



Figura 42 – Dashboard de saúde financeira - previsão de variação de caixa

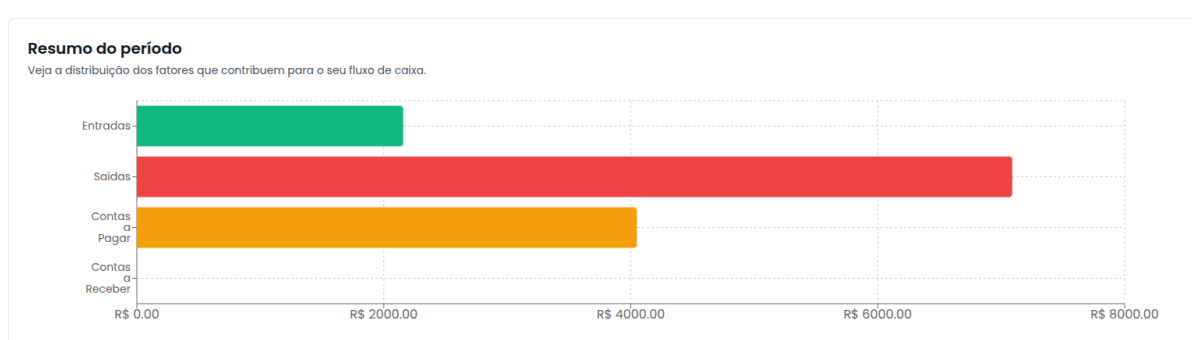
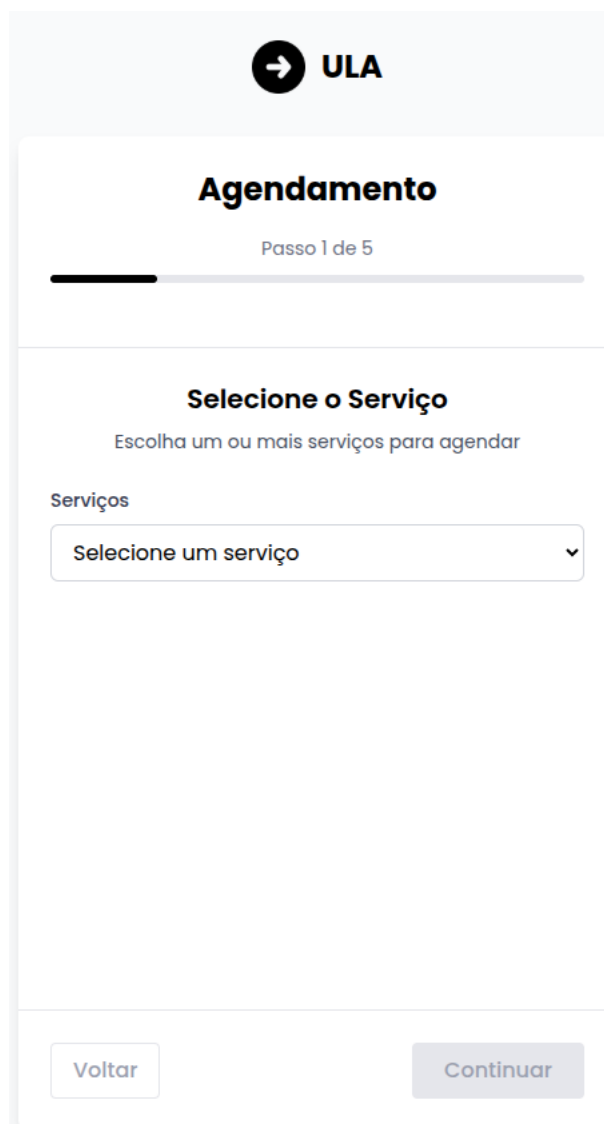


Figura 43 – Dashboard de saúde financeira - visão geral dos elementos que compõem a saúde financeira

Essas são as principais telas e funcionalidades que são oferecidas pelo projeto com as quais pretende-se proporcionar maior organização e facilidade ao gerente e próprio dono de negócio permitindo que ele consiga gerir seu empreendimento de forma simples conseguindo maiores lucros e fluidez.

Do ponto de vista de tecnologia e engenharia, ao agendar um serviço cria-se automaticamente uma conta a receber e assim que o agendamento é confirmado cria-se uma entrada de caixa, ou seja, cada tela além de possuir sua função própria está fortemente ligada a todas as outras telas e funcionalidades.

Além dessas telas, tem-se também uma interface para os clientes do empreendimento que vai poder agendar um serviço selecionando os serviços que deseja, dia, funcionário que irá realizar tais serviços e horário disponível. Os clientes acessam essa interface através do link para a plataforma mencionado anteriormente na seção de cadastro de clientes. Para realizar a integração total da plataforma bem como manter dar sentido e personalização à automação é necessário que esse link contenha a informação do identificador do cliente que iniciou a conversa, por se tratar de um dado sensível e evitar que pessoas má-intencionadas criem agendamentos na plataforma em nome de terceiros ao invés de expor o identificador real criou-se um token criptografado através de um algoritmo conhecido por AES em modo ECB com padding PKCS5, adicionando uma camada de segurança. Nesse procedimento converte-se o identificador em bytes e aplica-se a cifra com uma chave simétrica secreta para então codificar em base64 URL-safe garantindo compatibilidade com URLs. Após o cliente finalizar o agendamento o sistema descriptografa o token a fim de recuperar o identificador do cliente para cadastrar em seu nome os serviços requisitados.



The image shows a mobile application interface for scheduling services. At the top, there is a header with a black circle containing a white right-pointing arrow and the text "ULA" in bold. Below this, the main title "Agendamento" is displayed in bold. Underneath the title, it says "Passo 1 de 5" with a progress bar that is one-fifth full. The next section is titled "Selecione o Serviço" in bold, followed by the instruction "Escolha um ou mais serviços para agendar". Below this, there is a label "Serviços" and a dropdown menu with the text "Selecione um serviço" and a downward arrow. At the bottom of the screen, there are two buttons: "Voltar" (Back) on the left and "Continuar" (Continue) on the right.

→ ULA

Agendamento

Passo 1 de 5

Selecione o Serviço

Escolha um ou mais serviços para agendar

Serviços

Selecione um serviço ▼

Voltar Continuar

Figura 44 – Seleção de serviços para agendamento

ULA

Agendamento

Passo 2 de 5

Escolha a Data

Selecione o dia que você quer ser atendido

Data

28/09/2025

Voltar Continuar

Figura 45 – Escolha da data para agendar os serviços



The image shows a mobile application interface for scheduling a service. At the top, there is a header with a right arrow icon and the text "ULA". Below this, the main title "Agendamento" is displayed in bold. Underneath the title, it says "Passo 3 de 5" with a progress bar that is approximately 60% full. The section is titled "Escolha o Profissional" and contains the instruction: "Selecione um profissional ou deixe em branco para ver horários de todos os profissionais capacitados para o serviço". Below this instruction, there is a label "Funcionário" and a dropdown menu. The dropdown menu currently shows "João Silva" with a downward arrow icon. At the bottom of the screen, there are two buttons: "Voltar" (Back) on the left and "Continuar" (Continue) on the right.

ULA

Agendamento

Passo 3 de 5

Escolha o Profissional

Selecione um profissional ou deixe em branco para ver horários de todos os profissionais capacitados para o serviço

Funcionário

João Silva ▼

Voltar Continuar

Figura 46 – Escolha do profissional que irá realizar o serviço

The image shows a mobile application interface for scheduling an appointment. At the top, there is a header with a right arrow icon and the text "ULA". Below this, the main title "Agendamento" is displayed in bold. Underneath the title, it says "Passo 4 de 5" with a progress bar that is approximately 75% full. The section is titled "Selecione o Horário" (Select the Time) with the instruction "Escolha o horário que você gostaria de ser atendido" (Choose the time you would like to be attended). There is a list of four time slots, each with a label and the name "employeeA":

- 07:00 - 08:00
- 08:00 - 09:00 (This slot is highlighted with a blue border and background)
- 11:00 - 12:00
- 14:00 - 15:00

At the bottom of the screen, there are two buttons: "Voltar" (Back) on the left and "Continuar" (Continue) on the right.

Figura 47 – Escolha de um horário disponível

 ULA

Agendamento

Passo 5 de 5

Resumo do Agendamento

Confirme os dados do seu agendamento

Serviços	
Corte de cabelo	R\$ 50,00
Total	R\$ 50,00

Data e Horário

segunda-feira, 20 de outubro de 2025
08:00 - 09:00

Profissional

employeeA

Voltar

Confirmar Agendamento

Figura 48 – Resumo do agendamento atual

5 Considerações Finais

5.1 Conclusões do Projeto de Formatura

O presente trabalho propôs-se a desenvolver uma plataforma digital para gerenciamento de pequenos negócios, estabelecendo como critérios norteadores alinhamento com necessidades de uma parcela da população, potencial de continuidade como empresa e potencial impacto social. A análise dos resultados obtidos permite avaliar em que medida esses objetivos foram alcançados e quais limitações ainda persistem.

Do ponto de vista funcional, a plataforma implementada atende aos requisitos levantados inicialmente. As funcionalidades foram concretizadas e integradas, conforme demonstrado no Capítulo 4. Apesar do resultado positivo, destaca-se que a arquitetura implementada admite expansão ao abrir espaço para novos módulos no futuro, sem necessidade de grandes mudanças infraestruturais.

Quanto ao critério de alinhamento com as necessidades da população, os dados apresentados no Capítulo 2 corroboram a existência do problema: 40% das pequenas empresas não sobrevivem além de 5 anos, principalmente devido ao descontrole na gestão e falta de planejamento. A solução proposta endereça aspectos relevantes desse problema ao automatizar o controle de agenda e registros financeiros. Contudo, a validação efetiva dessa aderência demandaria testes com usuários reais em ambiente de produção, etapa não contemplada no escopo deste trabalho. Sem essa validação ainda não é possível afirmar que a solução é adequada às necessidades e ao perfil técnico dos pequenos empreendedores brasileiros.

O potencial de continuidade como empresa, segundo critério estabelecido, depende fundamentalmente da capacidade da plataforma de gerar valor suficiente para sustentar um modelo de negócio viável. A arquitetura implementada — baseada em microsserviços com comunicação gRPC e separação lógica de dados — demonstra preocupação com escalabilidade técnica. Todavia, aspectos críticos para a viabilidade comercial permanecem indefinidos: modelo de precificação, estratégia de go-to-market, custos de infraestrutura em escala, e mecanismos de retenção de usuários.

Relativamente ao impacto social esperado, a plataforma possui potencial de contribuir para a melhora na saúde financeira de pequenos negócios ao oferecer ferramentas de gestão acessíveis. Tais aspectos estão presentes na solução elaborada, sendo que sua efetividade ainda deve ser testada com usuários reais conforme mencionado anteriormente.

5.2 Perspectivas de continuidade

A arquitetura implementada segue boas práticas definidas pela engenharia de software, bem como estabelece fundamentos que facilitam a evolução incremental do sistema rumo a uma plataforma digital completa. A modularidade inerente à abordagem de microsserviços permite que novos domínios de negócio sejam incorporados sem alterações disruptivas na infraestrutura existente. Esta característica é particularmente relevante considerando-se a diversidade de tipos de pequenos negócios que a plataforma pretende atender. Ademais, a arquitetura modular implementada torna trivial e retrocompatível a criação de novos módulos, por exemplo o módulo de integração com inteligência artificial, idealizado e discutido nos capítulos iniciais.

A expansão horizontal da plataforma para atender diferentes segmentos de pequenos negócios demandará a implementação de configurabilidade avançada, devido à heterogeneidade dos pequenos negócios existentes no Brasil. A arquitetura atual abstrai tais complexidades ao encapsular regras de negócio específicas dentro de módulos genéricos integrados ao núcleo da plataforma. Dessa forma, a flexibilidade arquitetural pode ser complementada por um sistema de configuração que permita aos próprios empreendedores adaptarem a plataforma às peculiaridades de seus negócios sem necessidade de desenvolvimento customizado.

A estratégia de persistência baseada em schemas lógicos compartilhando uma instância física de PostgreSQL, embora pragmática para a fase atual, apresenta limitações inerentes de escalabilidade. Em cenários de crescimento significativo da base de usuários, a migração para bancos de dados fisicamente separados por microsserviço seria necessária, mas a separação lógica implementada garante que este processo não afetaria a lógica de negócios criada na plataforma.

Do ponto de vista de negócios, a viabilização da plataforma como empresa requer definições estratégicas como monetização e aquisição de usuários, bem como testes com uma base de pequenos empreendedores reais, a fim de se entender como a plataforma se encaixa na rotina dessas pessoas e onde esta pode ser melhorada.

Através da arquitetura consolidada será plausível sem grandes dificuldades complementar a plataforma com requisitos que podem trazer mais robustez para a solução apresentada, por exemplo, o próprio cadastro de empresas, transformando nossa solução em uma plataforma completa com diversos pequenos negócios como clientes. Alinhado a isso deve-se implementar sistemas para controlar acesso a cada um dos sites e seções para diferentes funcionários do empreendimento, que foram abstraídas no Projeto de Formatura devido ao escopo a ser apresentado, contudo existe a ideia de descentralizar esse serviço utilizando uma integração a um serviço de emails para criar usuário e senha, garantindo maior sigilo e controle de acesso, o email hoje já é cadastrado no banco de dados ao

registrar um funcionário.

Devido a problemas burocráticos que impediram o uso da API oficial da Meta, empresa dona do aplicativo WhastApp, como a falta de uma empresa real por trás do projeto e CNPJ, que são necessários para conectar essa API a um número real, foi utilizada uma segunda opção que possui o mesmo papel dentro do escopo da solução apresentada que seria o da comunicação entre a plataforma e o aplicativo, a API conhecida por Maytapi, mas ela apenas deve ser utilizada em projetos, ou seja, a migração deve ser feita para expandir o projeto ou no caso da plataforma ser comercializada. Na mesma linha, por conta de limite de crédito que a AWS fornece no primeiro acesso e limitações da conta grátis apenas temos um endereço HTTP e não HTTPS, mas o Google, utilizado no celular, ao acessar esse endereço por padrão muda para HTTPS e por isso apesar do cliente atualmente receber um link clicável precisará alterar o endereço da página (voltando para HTTP), tratando-se de outra melhoria obrigatória.

Outra funcionalidade que deve melhorar a imersão da plataforma no controle do pequeno negócio é a integração com o serviço de WhatsApp buscando que o cliente confirme ou cancele um agendamento, realize pagamentos, envie comprovantes de pagamento, seja utilizado como meio de divulgação de promoções e entre outras funcionalidades que devem aproximar o empreendimento do cliente projetando maiores receitas e lucros, mas principalmente proporcionar um atendimento personalizado e objetivo otimizando o tempo do cliente e do gerente.

Em suma, a arquitetura implementada provê fundações adequadas para a evolução, tornando o projeto uma plataforma digital completa e comercialmente viável.

5.3 Do curso de engenharia

O desenvolvimento deste projeto de formatura proporcionou a aplicação prática de um espectro abrangente de conhecimentos adquiridos ao longo do curso de Engenharia de Computação na Escola Politécnica da USP, permitindo consolidar habilidades técnicas e exercitar competências de engenharia que transcendem a implementação de código.

A especificação arquitetural baseada no RM-ODP, framework estudado em disciplinas de sistemas de computação, demonstrou-se particularmente valiosa ao estruturar a decomposição do sistema em múltiplas perspectivas complementares, bem como a adaptação de frameworks teóricos para o escopo do nosso projeto.

A implementação de uma arquitetura distribuída baseada em microsserviços com comunicação gRPC demandou a aplicação de conceitos de sistemas operacionais, redes de computadores, e engenharia de software. Tais decisões exigiram a compreensão das restrições envolvidas ao escolher uma solução em detrimento de outra. Da mesma forma,

a organização do código e a separação modular entre componentes frontend, backend, e bibliotecas compartilhadas aplicaram conhecimentos de engenharia de software sobre gerenciamento de dependências, versionamento, e estruturação de projetos de larga escala.

O exercício de engenharia de requisitos evidenciou a complexidade de traduzir problemas do mundo real em especificações técnicas precisas. A tensão entre a amplitude das necessidades identificadas e as restrições de tempo e recursos para desenvolvimento exemplifica desafios típicos de projetos de engenharia, nos quais priorização e gestão de escopo são tão críticas quanto competência técnica.

A experiência de trabalho em equipe ao longo de um projeto de duração estendida proporcionou aprendizados sobre dinâmica de colaboração, comunicação técnica, e resolução de conflitos que são raramente explicitados em ambientes acadêmicos tradicionais mas são absolutamente críticos em contextos profissionais. A necessidade de coordenar esforços de desenvolvimento entre três pessoas, manter coerência arquitetural, e integrar componentes desenvolvidos independentemente replicou desafios típicos de ambientes industriais de desenvolvimento de software.

Em retrospectiva, o projeto proporcionou consolidação de habilidades técnicas fundamentais, enquanto exercitou aspectos da formação teórica em engenharia ao exigir fundamentos conceituais sólidos e capacidade de aprendizado autônomo que foram essenciais para o projeto. Esta conclusão crítica sobre competências adquiridas e áreas que demandam aprofundamento constitui o resultado mais valioso do trabalho realizado, explicitando onde precisamos nos desenvolver para nos tornarmos engenheiros melhores.

Referências

- ABREU, C. *Pequenos negócios são quase a totalidade das empresas abertas no Brasil este ano*. 2024. ASN Nacional - Agência Sebrae de Notícias. Disponível em: <<https://agenciasebrae.com.br/dados/pequenos-negocios-sao-quase-a-totalidade-das-empresas-abertas-no-brasil-este-ano/>>. Citado na página 15.
- Amazon Web Services. *Amazon Web Services (AWS) - Cloud Computing Services*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://aws.amazon.com>>. Citado na página 42.
- ASADULLAH, A.; FAIK, I.; KANKANHALLI, A. Digital platforms: a review and future directions. 2023. Disponível em: <https://repository.lboro.ac.uk/articles/conference_contribution/Digital_platforms_a_review_and_future_directions/24081825>. Citado na página 12.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software Architecture in Practice*. 4. ed. Boston, MA: Addison-Wesley Professional, 2021. ISBN 978-0-13-668963-2. Citado na página 26.
- DIAS, M. C. *Cinco dados que comprovam a importância dos pequenos negócios para o Brasil*. 2024. CNN Brasil. Disponível em: <<https://www.cnnbrasil.com.br/economia/negocios/cinco-dados-que-comprovam-a-importancia-dos-pequenos-negocios-para-o-brasil/>>. Citado na página 15.
- Docker Inc. *Docker: Accelerated Container Application Development*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://www.docker.com>>. Citado na página 42.
- Docker Inc. *Docker Compose: Define and run multi-container applications*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://docs.docker.com/compose>>. Citado na página 42.
- EMPREENDEDORISMO no Brasil tem atingido milhões de pessoas. 2024. Valor Econômico. Disponível em: <<https://valor.globo.com/patrocinado/dino/noticia/2024/04/10/empreendedorismo-no-brasil-tem-atingido-milhoes-de-pessoas.ghtml>>. Citado na página 15.
- EVANS, E. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley Professional, 2003. ISBN 0-321-12521-5. Citado na página 31.
- GIBSON, K. *Digital Platforms: What They Are & How They Create Value*. 2024. Business Insights Blog. Disponível em: <<https://online.hbs.edu/blog/post/what-is-a-digital-platform>>. Citado 2 vezes nas páginas 12 e 13.
- GLEISON, B.; RICARDO, T.; VALENTE, M. T. *Monorepos: A Multivocal Literature Review*. 2018. Disponível em: <<http://arxiv.org/abs/1810.09477>>. Citado na página 38.
- gRPC Authors. *gRPC: A high performance, open source universal RPC framework*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://grpc.io>>. Citado na página 35.

- HARTMANS, A. *Airbnb now has more listings worldwide than the top five hotel brands combined*. 2017. Business Insider. Disponível em: <<https://www.businessinsider.com/airbnb-total-worldwide-listings-2017-8>>. Citado na página 13.
- HEIN, A. et al. Digital platform ecosystems. *Electronic Markets*, v. 30, n. 1, p. 87–98, 2020. Citado na página 13.
- ISO/IEC. *ISO/IEC 10746:2009 - Information technology - Open Distributed Processing - Reference Model*. Geneva, Switzerland, 2009. Citado na página 16.
- ISO/IEC. *ISO/IEC 25010:2011 - Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models*. Geneva, Switzerland, 2011. Citado na página 21.
- ISO/IEC. *ISO/IEC 12207:2017 - Systems and software engineering - Software life cycle processes*. Geneva, Switzerland, 2017. Citado na página 38.
- JEBBAR, M.; SEKKAKI, A.; BENAMAR, O. Integration of soa and cloud computing in rm-odp. In: *IEEE*. [s.n.], 2012. Disponível em: <<https://ieeexplore.ieee.org/document/6481897>>. Citado 2 vezes nas páginas 16 e 17.
- JetBrains s.r.o. *Kotlin Programming Language*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://kotlinlang.org>>. Citado 2 vezes nas páginas 37 e 39.
- MÁRQUEZ, G.; ASTUDILLO, H.; KAZMAN, R. Architectural tactics in software architecture: A systematic mapping study. *Journal of Systems and Software*, v. 197, p. 111558, 2023. Citado na página 26.
- Meta Platforms, Inc. *React: The library for web and native user interfaces*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://react.dev>>. Citado 2 vezes nas páginas 37 e 41.
- Microsoft Corporation. *TypeScript: JavaScript With Syntax For Types*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://www.typescriptlang.org>>. Citado 2 vezes nas páginas 37 e 41.
- NARDES, F. B. S.; MIRANDA, R. C. d. R. Lean startup e canvas: uma proposta de metodologia para startups. *Revista Brasileira de Administração Científica*, v. 5, n. 3, p. 252–272, 2014. Citado na página 19.
- OSTERWALDER, A.; PIGNEUR, Y. Modeling value propositions in e-business. In: *Proceedings of the 5th international conference on Electronic commerce*. New York, NY, USA: Association for Computing Machinery, 2003. (ICEC '03), p. 429–436. Disponível em: <<https://dl.acm.org/doi/10.1145/948005.948061>>. Citado na página 19.
- PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. 2024. Acesso em: 19 nov. 2024. Disponível em: <<https://www.postgresql.org>>. Citado 2 vezes nas páginas 37 e 42.
- PRESSMAN, R. S.; MAXIM, B. R. *Software Engineering: A Practitioner's Approach*. 8. ed. New York, NY: McGraw-Hill Education, 2014. ISBN 978-0-07-802212-8. Citado na página 21.

RAYMOND, K. Reference model of open distributed processing (rm-odp): Introduction. In: *Open Distributed Processing*. Boston, MA: Springer US, 1995. p. 3–14. Disponível em: https://www.academia.edu/114524166/Open_Distributed_Processing. Citado 2 vezes nas páginas 16 e 17.

REUVER, M. d.; SØRENSEN, C.; BASOLE, R. C. The digital platform: A research agenda. 2018. Disponível em: <https://journals.sagepub.com/doi/abs/10.1057/s41265-016-0033-3>. Citado na página 13.

STEEN, M. W. A.; DERRICK, J. Odp enterprise viewpoint specification. *Computer standards & interfaces*, v. 22, n. 3, p. 165–189, 2000. Disponível em: <https://www.sciencedirect.com/science/article/abs/pii/S0920548900000313>. Citado na página 17.

UM país de 90 milhões de empreendedores. 2024. ASN Nacional - Agência Sebrae de Notícias. Disponível em: <https://agenciasebrae.com.br/dados/um-pais-de-90-milhoes-de-empreendedores/>. Citado na página 15.